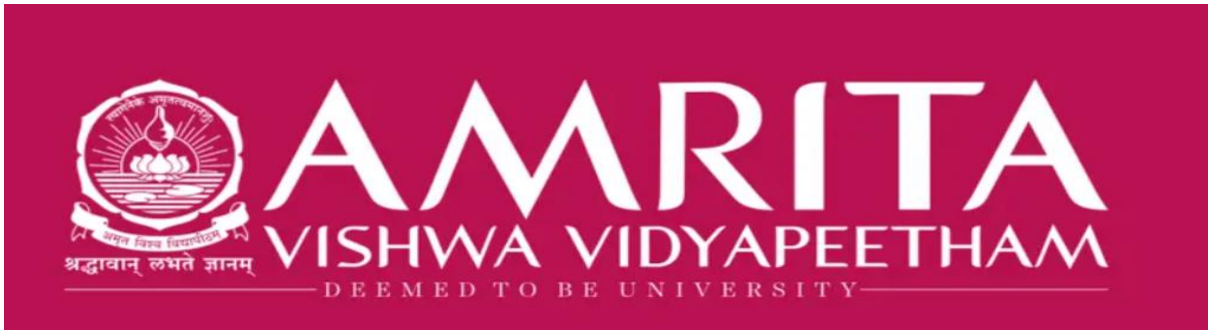


DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	P. Bharath Chand
ROLL NO	AV.SC.U4CSE25225
YEAR/SEM/SECTION	2ND YEAR/SEM-3/CSE-C
CREDITS	05

SUBMITTED TO	Dr Gayathri Parasa
DESIGNATION	Asst. Professor (Sr. Gr)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO
1.	**WEEK 1:** **1(A).** A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!". **1(B).** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years. **1(C).** A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **1(D).** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. **1(E).** Write a recursive python program to print the first n terms of the Fibonacci series.	
2.	**WEEK 2:** **2(A).** Write a python program to implement linear search. **2(B).** Write a python program to implement binary search with and without sorted array as an input.	
3.	**WEEK-3:** **3(A).** Write a python program to implement bubble sort. **3(B).** Write a python program to implement insertion sort. **3(C).** Write a python program to implement selection sort.	

4.	**WEEK-4:** **4(A).** Write a python program to implement quick sort. **4(B).** Write a python program to implement merge sort.	
5.	**WEEK-5:** **5(A).** Write a python program to implement a Singly Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Sort the data of SLL 10) Search data in SLL 11) Display or Traverse 12) Exit	
6.	**WEEK-6:** Write a python program to implement a Double Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value	

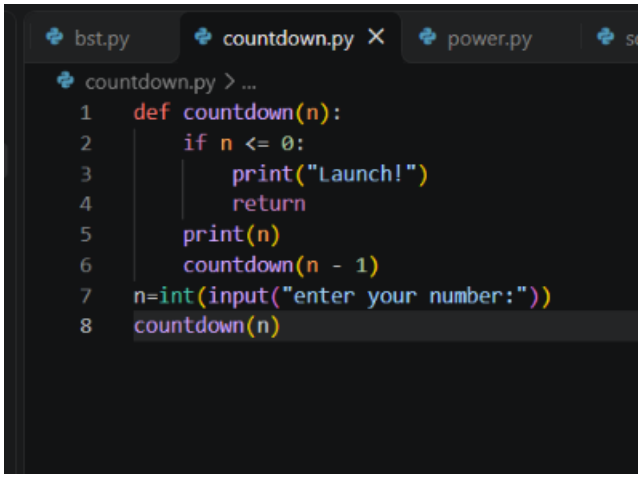
	6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Display or Traverse 10) Exit	
7.	**WEEK-7:** **7(A).** Write a python program to implement stack operations such as push, pop, peek, display using an array. **7(B).** Write a python program to implement stack operations such as push, pop, peek, display using a linked list.	
8.	**WEEK-8:** **8(A).** Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. **8(B).** Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.	
9.	**WEEK-9:** Write a python program to implement a Binary Search Tree (BST) and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals – in-order, pre-order and post-order 4. Count Leaf Nodes 5. Count Internal Nodes	

	6. Height of the Tree	
10.	**WEEK-10:** Write a menu driven python program to implement an AVL Tree and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	
11.	**WEEK-11:** Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations: 1. Insertion of an element into the Heap. 2. Deletion of the root element from the Heap. 3. Display the array representation of the Heap. 4. Exit the program.	
12.	**WEEK-12:** Write a program to implement the following three collision resolution techniques in a Hash Table: 1. Separate Chaining 2. Linear Probing 3. Quadratic Probing	

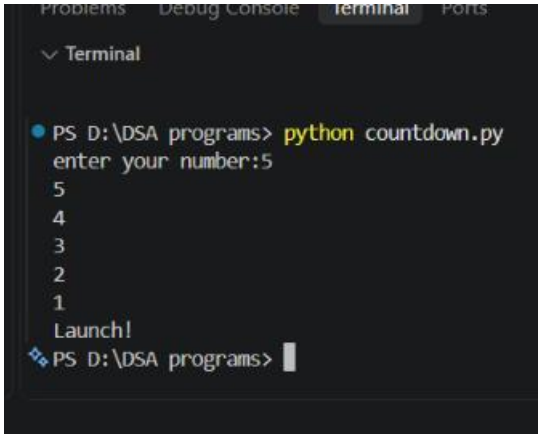
WEEK 1:

AIM: 1(A). A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!".

CODE:



Output:



1(B). Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P n(power) where: P= principal growth factor, n= number of years.

```python
def power(p, n):
    # Base case: anything to the power of 0 is 1
    if n == 0:
        return 1
    # Recursive step
    return p * power(p, n - 1)
p=int(input("enter the base:"))
n=int(input("enter the exponent:"))
print(power(p, n))
```

Output:

```
PS D:\DSA programs> python power.py
enter the base:5
enter the exponent:2
25
PS D:\DSA programs>
```

1(C). A HR department stores employee ID ’s in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

CODE:

```python
def search_employee(emp_list, target_id, index=0):
    # Base case: if we reach the end of the list
    if index == len(emp_list):
        return -1

    # Base case: if the element is found
    if emp_list[index] == target_id:
        return index

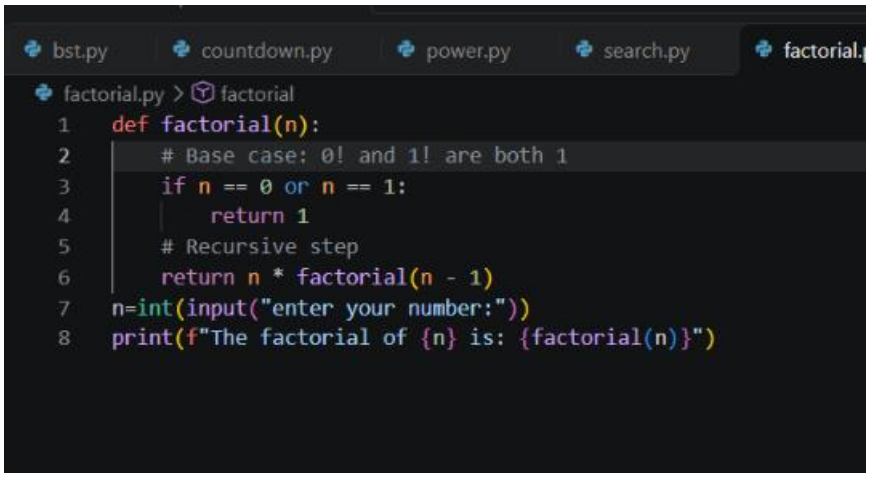
    # Recursive step: check the next index
    return search_employee(emp_list, target_id, index + 1)
list=[101, 102, 103, 104, 105]
target_id=int(input("Enter the employee ID to search: "))
result = search_employee(list, target_id)
if result != -1:
    print(f"Employee found at index {result}")
else:
    print("Employee not found")
```

Output:

```
PS D:\DSA programs> python search.py
Enter the employee ID to search: 101
Employee found at index 0
PS D:\DSA programs>
```

1(D). In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

CODE:



Output:

```
PS D:\DSA programs> python factorial.py
enter your number:5
The factorial of 5 is: 120
PS D:\DSA programs>
```

1(E). Write a recursive python program to print the first n terms of the Fibonacci series.

Code:

bst.py | countdown.py | power.py | search.py | factorial.py

fibonnaci.py > print_fib_series

```python
def fibonacci(i):
    if i <= 1:
        return i
    return fibonacci(i - 1) + fibonacci(i - 2)

def print_fib_series(n, current=0):
    if current == n:
        return
    print(fibonacci(current), end=" ")
    print_fib_series(n, current + 1)
n=int(input("Enter the number of terms for Fibonacci series: "))
print_fib_series(n)
```

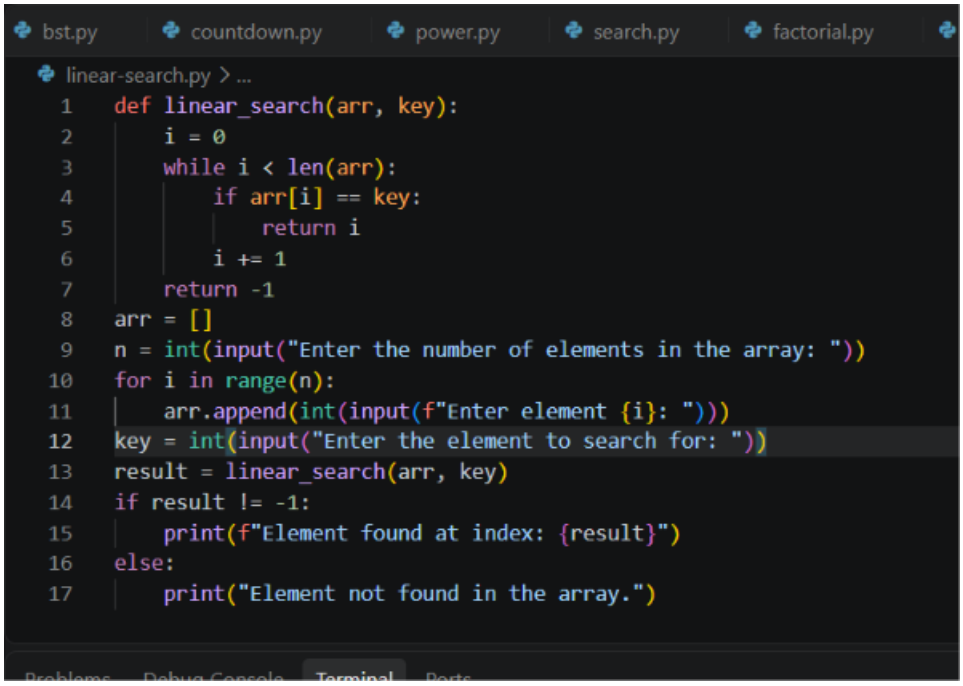
Output:

```
PS D:\DSA programs> python fibonnaci.py
Enter the number of terms for Fibonacci series: 5
0 1 1 2 3
PS D:\DSA programs>
```

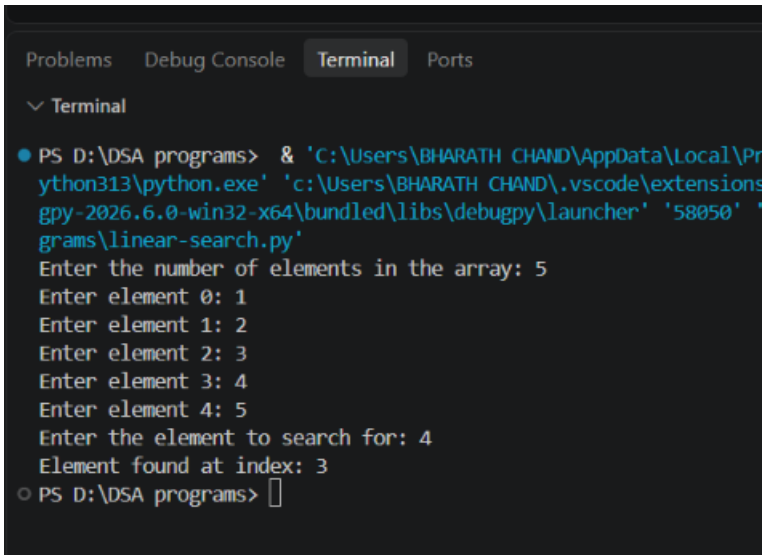
WEEK 2:

AIM: 2(A). Write a python program to implement linear search.

CODE:

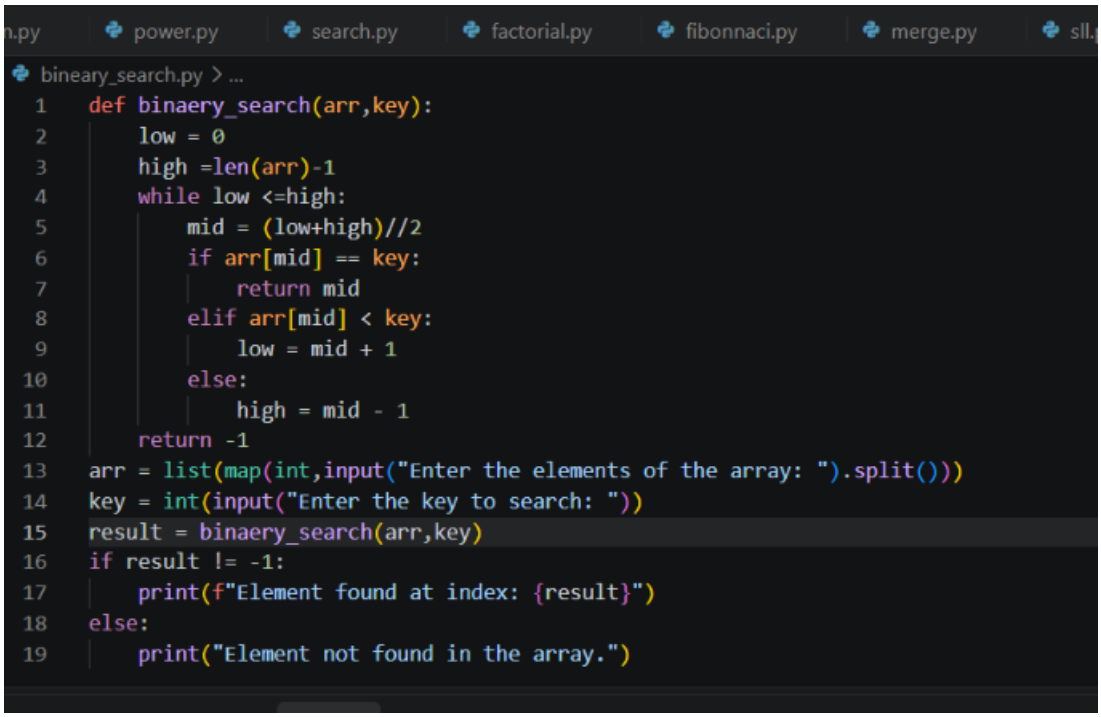


Output:



2(B). Write a python program to implement binary search with and without sorted array as an input. CODE:

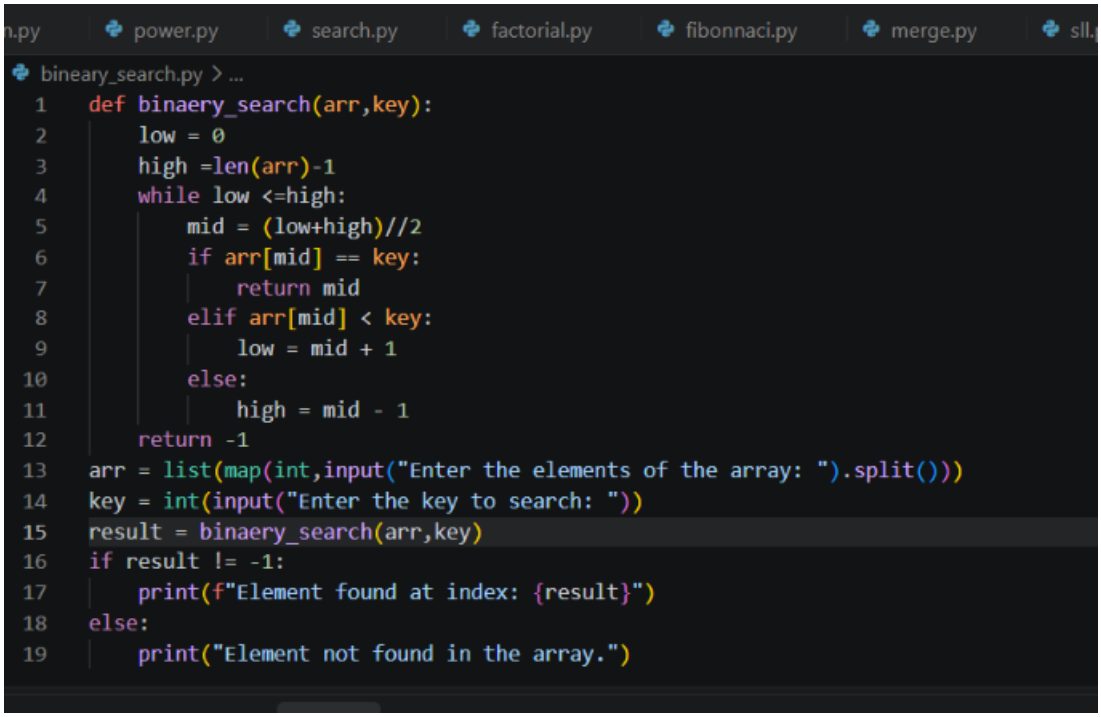
Without Sorted Array:



Output:



With Sorted Array:

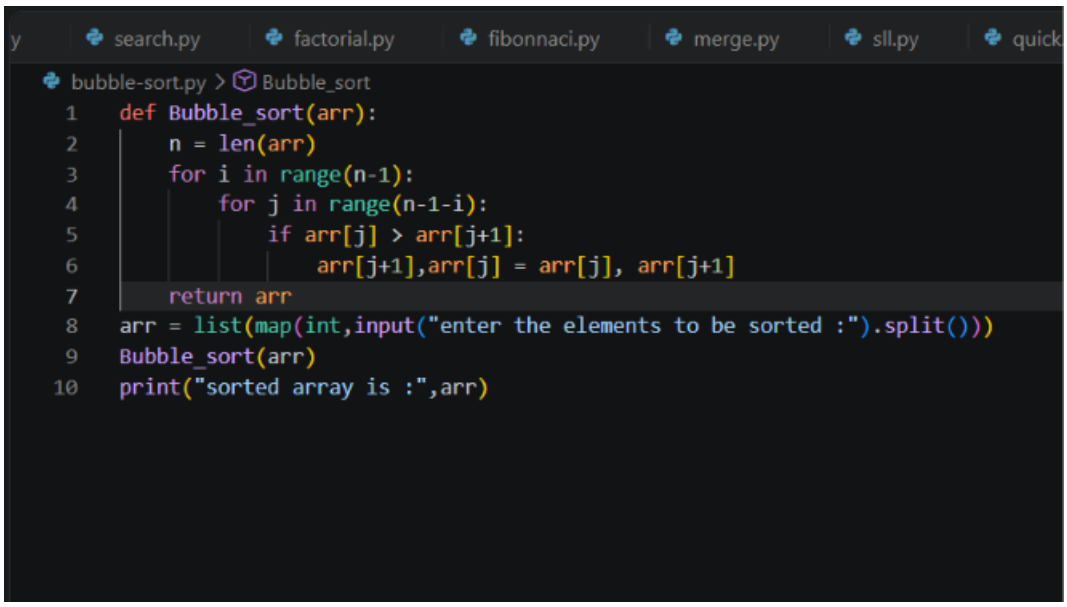


Output:



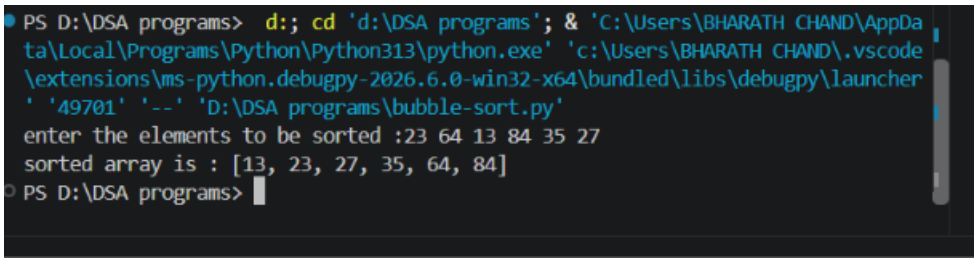
WEEK-3: AIM: 3(A). Write a python program to implement bubble sort.

CODE:



```
bubble-sort.py > Bubble_sort
def Bubble_sort(arr):
    n = len(arr)
    for i in range(n-1):
        for j in range(n-1-i):
            if arr[j] > arr[j+1]:
                arr[j+1],arr[j] = arr[j], arr[j+1]
    return arr
arr = list(map(int,input("enter the elements to be sorted :").split()))
Bubble_sort(arr)
print("sorted array is :",arr)
```

Output:



```
PS D:\DSA programs>  d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHAND\AppDa
ta\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscode
\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher
' '49701' '--' 'D:\DSA programs\bubble-sort.py'
enter the elements to be sorted :23 64 13 84 35 27
sorted array is : [13, 23, 27, 35, 64, 84]
PS D:\DSA programs>
```

3(B). Write a python program to implement insertion sort.

CODE:



```
insertion_sort.py > ...
def insertion_sort(arr):
    n=len(arr)
    for i in range(n):
        key=arr[i]
        j=i-1
        while j>=0 and arr[j]>key:
            arr[j+1]=arr[j]
            j-=1
        arr[j+1]=key
    return arr
arr = list(map(int,input("enter the elements to be sorted :").split()))
insertion_sort(arr)
print("sorted array is :",arr)
```

Output:

```
PS D:\DSA programs>  d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHAND\AppD
ta\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscod
\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launche
' '50703' '--' 'D:\DSA programs\insertion_sort.py'
enter the elements to be sorted :34 74 13 84 35 72
sorted array is : [13, 34, 35, 72, 74, 84]
PS D:\DSA programs>
```

3(C). Write a python program to implement selection sort.

CODE:

fibonnaci.py | merge.py | sll.py | quick.py | linear-search.py | bineary_search

selection.py > selection_sort

```python
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        if min_index != i:
            arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr

arr = list(map(int,input("enter the elements to be sorted :").split()))
selection_sort(arr)
print("sorted array is :",arr)
```

Output:

```
PS D:\DSA programs>  d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHAND\AppDa
ta\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscode
\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher
' '59037' '--' 'D:\DSA programs\selection.py'
enter the elements to be sorted :45 12 73 2 4 98 5 6
sorted array is : [2, 4, 5, 6, 12, 45, 73, 98]
PS D:\DSA programs>
```

WEEK-4:

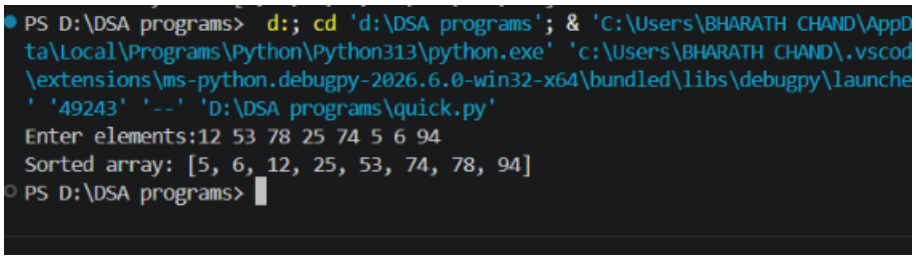
AIM: (A). Write a python program to implement quick sort.

CODE:



```python
def quick(arr,low,high):
    if low<high:
        i=low
        j=high
        pivot=low
        while i<j:
            while arr[i]<=arr[pivot] and i<high:
                i+=1
            while arr[j]>arr[pivot]:
                j-=1
            if i<j:
                arr[i],arr[j]=arr[j],arr[i]
        arr[pivot],arr[j]=arr[j],arr[pivot]
        quick(arr,low,j-1)
        quick(arr,j+1,high)
arr=list(map(int,input("Enter elements:").split()))
quick(arr,0,len(arr)-1)
print("Sorted array:",arr)
```

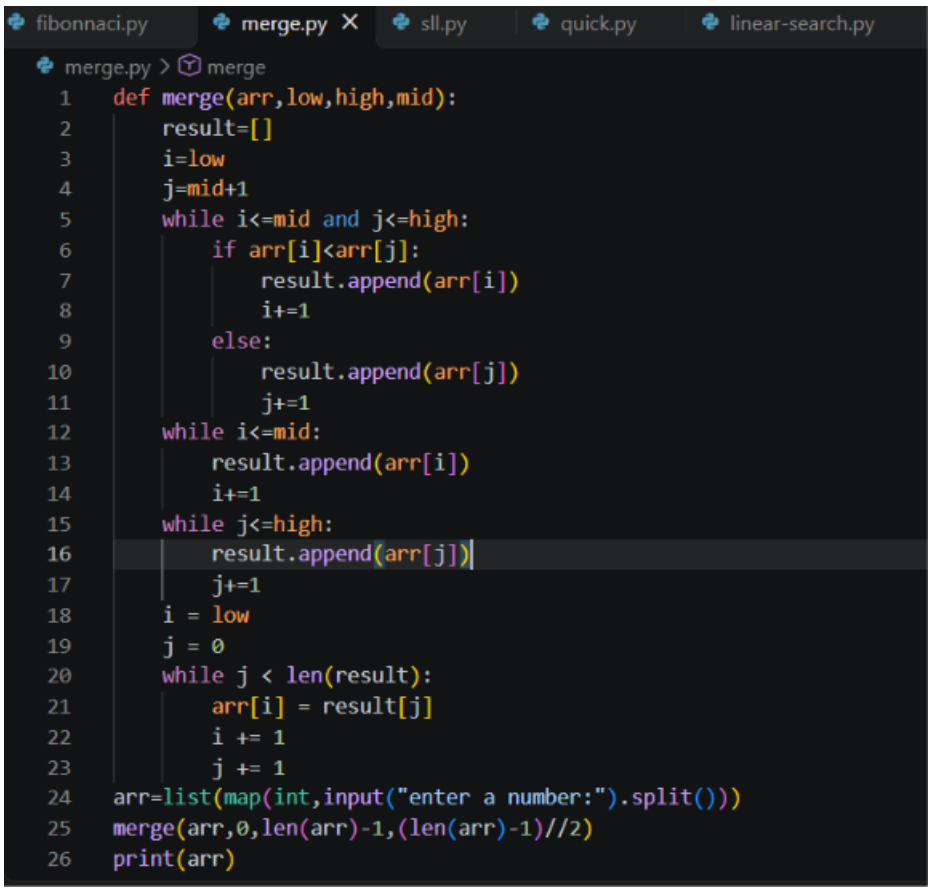
Output:



```
PS D:\DSA programs>  d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHAND\AppD
ta\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscod
\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launche
' '49243' '--' 'D:\DSA programs\quick.py'
Enter elements:12 53 78 25 74 5 6 94
Sorted array: [5, 6, 12, 25, 53, 74, 78, 94]
PS D:\DSA programs>
```

(B).Write a python program to implement merge sort.

CODE:



```
fibonnaci.py  merge.py  sll.py  quick.py  linear-search.py
merge.py > merge
def merge(arr,low,high,mid):
    result=[]
    i=low
    j=mid+1
    while i<=mid and j<=high:
        if arr[i]<arr[j]:
            result.append(arr[i])
            i+=1
        else:
            result.append(arr[j])
            j+=1
    while i<=mid:
        result.append(arr[i])
        i+=1
    while j<=high:
        result.append(arr[j])
        j+=1
    i = low
    j = 0
    while j < len(result):
        arr[i] = result[j]
        i += 1
        j += 1
arr=list(map(int,input("enter a number:").split()))
merge(arr,0,len(arr)-1,(len(arr)-1)//2)
print(arr)
```

Output:



```
PS D:\DSA programs>  d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHAND\AppDa
ta\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscode
\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher
' '63031' '--' 'D:\DSA programs\merge.py'
enter a number:45 67 23 84 12 34
[45, 67, 23, 84, 12, 34]
PS D:\DSA programs>
```

WEEK – 5 AIM:

(A): Write a python program to implement a Singly Linked List with the below operations.

1) Create Linked List

2) Insert Beginning

3) Insert End

4) Insert at Index

5) Delete by Value

6) Delete First Node

7) Delete Last Node

8) Count number of nodes

9) Sort the data of SLL

10) Search data in SLL

11) Display or Traverse

12)Exit

fibonnaci.py | merge.py | sll.py | quick.py | linear-search.py | bi

sll.py > ...

```python
class Node:
    def __init__(self,data):
        self.data=data
        self.next=None
class sll:
    def __init__(self):
        self.head=None
    def insert_b(self,data):
        new=Node(data)
        new.next=self.head
        self.head=new
    def insert_e(self,data):
        new=Node(data)
        if self.head is None:
            self.head=new
        else:
            temp = self.head
            while temp.next:
                temp=temp.next
            temp.next=new
    def insert_a(self,data,index):
        new=Node(data)
        if self.head is None:
            self.head=new
        elif index==0:
            self.insert_b(data)
        elif index==self.count():
            self.insert_e(data)
        else:
            temp=self.head
            for i in range(index-1):
                temp=temp.next
            new.next=temp.next
```

fibonnaci.py | merge.py | sll.py | quick.py | linear-s

sll.py > ...

```
class sll:
    def count(self):
        if self.head is None:
            return 0
        else:
            temp=self.head
            c=0
            while temp:
                c+=1
                temp=temp.next
            return c
    def display(self):
        if self.head is None:
            return None
        else:
            temp=self.head
            while temp:
                print(temp.data,end=" ")
                temp=temp.next
            print()
    def delete_b(self):
        if self.head is None:
            return None
        else:
            self.head=self.head.next
            print("Deleted")
    def delete_e(self):
        if self.head is None:
            return None
        elif self.head.next is None:
            self.head=None
            print("Deleted")
        else:
```

fibonnaci.py | merge.py | sll.py | quick.py | linear-search.py | bineary_search.py | bubble-sort.py | insertion_sort.py | selection.py

sll.py > ...

```
class sll:
    def delete_a(self,index):
        if self.head is None:
            return None
        elif index==0:
            self.delete_b()
        elif index==self.count()-1:
            self.delete_e()
        else:
            temp=self.head
            for i in range(index-1):
                temp=temp.next
            temp.next=temp.next.next
            print("Deleted")
s=sll()
while True:
    print("1.Insert at beginning\n2.Insert at end\n3.Insert at any index\n4.Count\n5.Display\n6.Delete at beginning\n7.Delete at end\n8.Dele
    ch=int(input("Enter your choice:"))
    if ch==1:
        data=int(input("Enter data:"))
        s.insert_b(data)
    elif ch==2:
        data=int(input("Enter data:"))
        s.insert_e(data)
    elif ch==3:
        data=int(input("Enter data:"))
        index=int(input("Enter index:"))
        s.insert_a(data,index)
    elif ch==4:
        print(s.count())
    elif ch==5:
        s.display()
```

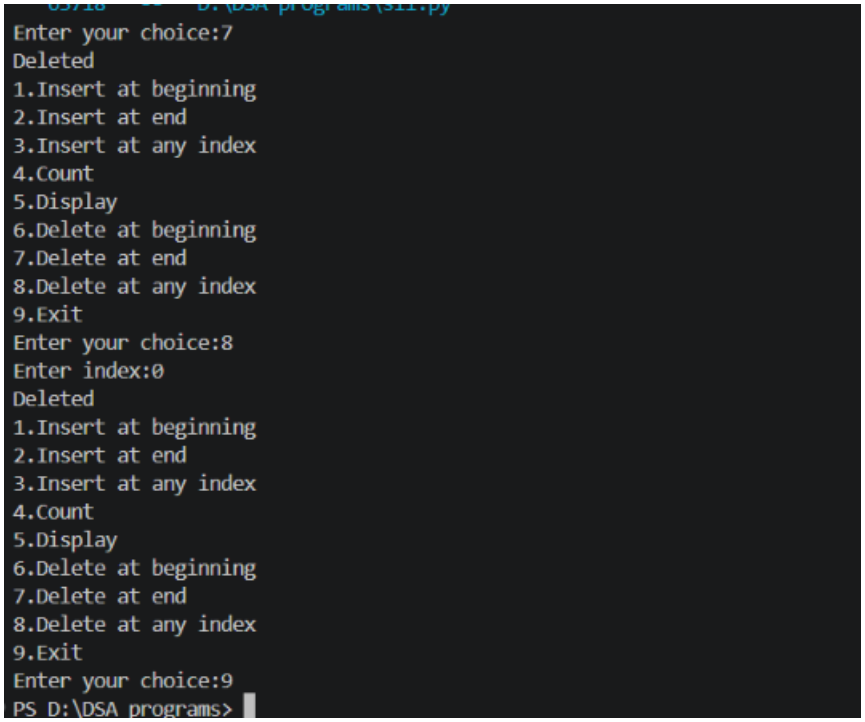


Output

```
PS D:\DSA programs>  d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHAND\AppDa
ta\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscode
\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher
' '63718' '--' 'D:\DSA programs\sll.py'
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
7.Delete at end
8.Delete at any index
9.Exit
Enter your choice:1
Enter data:10
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
7.Delete at end
8.Delete at any index
9.Exit
Enter your choice:2
Enter data:20
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
```

```
PS D:\DSA programs>  d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHAND\AppDa
ta\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscode
\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher
' '63718' '--' 'D:\DSA programs\sll.py'
9.Exit
Enter your choice:3
Enter data:30
Enter index:1
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
7.Delete at end
8.Delete at any index
9.Exit
Enter your choice:4
3
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
7.Delete at end
8.Delete at any index
9.Exit
```

```
Enter your choice:5
10 30 20
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
7.Delete at end
8.Delete at any index
9.Exit
Enter your choice:6
Deleted
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
7.Delete at end
8.Delete at any index
9.Exit
```


Enter your choice:7
Deleted
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
7.Delete at end
8.Delete at any index
9.Exit
Enter your choice:8
Enter index:0
Deleted
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
7.Delete at end
8.Delete at any index
9.Exit
Enter your choice:9
PS D:\DSA programs>

WEEK-6

AIM: Write a python program to implement a Double Linked List with the below operations.

1) Create Linked List

2) Insert Beginning

3) Insert End

4) Insert at Index

5) Delete by Value

6) Delete First Node

7) Delete Last Node

8) Count number of nodes

9) Display or Traverse

10)Exit


dll.py > dll > insert_a
class Node:
def __init__(self, data):
self.data = data
self.next = None
self.prev = None
class dll:
def __init__(self):
self.head = None
def insert_b(self, data):
new = Node(data)
if self.head is not None:
self.head.prev = new
new.next = self.head
self.head = new
def insert_e(self, data):
new = Node(data)
if self.head is None:
self.head = new
else:
temp = self.head
while temp.next:
temp = temp.next
temp.next = new
new.prev = temp
def insert_a(self, data, index):
new = Node(data)
if self.head is None:
self.head = new
elif index == 0:
self.insert_b(data)
elif index == self.count():
self.insert_e(data)

fibonnaci.py | merge.py | sll.py | dll.py | quick.py

dll.py > dll > insert_a

```
 7  class dll:
29      def insert_a(self, data, index):
37          else:
38              temp = self.head
39              for i in range(index - 1):
40                  temp = temp.next
41              new.next = temp.next
42              new.prev = temp
43              if temp.next:
44                  temp.next.prev = new
45              temp.next = new
46
47      def count(self):
48          if self.head is None:
49              return 0
50          else:
51              temp = self.head
52              c = 0
53              while temp:
54                  c += 1
55                  temp = temp.next
56              return c
57
58      def display(self):
59          if self.head is None:
60              return None
61          else:
62              temp = self.head
63              while temp:
64                  print(temp.data, end=" ")
65                  temp = temp.next
66              print()
67
68      def delete_b(self):
69          if self.head is None:
70              return None
```

fibonnaci.py | merge.py | sll.py | dll.py | quick.py

dll.py > dll > insert_a

```
  7   class dll:
 68       def delete_b(self):
 71           else:
 72               self.head = self.head.next
 73               if self.head is not None:
 74                   self.head.prev = None
 75               print("Deleted")
 76
 77       def delete_e(self):
 78           if self.head is None:
 79               return None
 80           elif self.head.next is None:
 81               self.head = None
 82               print("Deleted")
 83           else:
 84               temp = self.head
 85               while temp.next:
 86                   temp = temp.next
 87               temp.prev.next = None
 88               print("Deleted")
 89
 90       def delete_a(self, index):
 91           if self.head is None:
 92               return None
 93           elif index == 0:
 94               self.delete_b()
 95           elif index == self.count() - 1:
 96               self.delete_e()
 97           else:
 98               temp = self.head
 99               for i in range(index):
100                   temp = temp.next
101               temp.prev.next = temp.next
102               if temp.next:
103                   temp.next.prev = temp.prev
104               print("Deleted")
```

```python
s = dll()
while True:
    print("1.Insert at beginning\n2.Insert at end\n3.Insert at any index\n4.Count\n5.Display\n6.Delete at beginning\n7.Delete at end\n8.Dele
    ch = int(input("Enter your choice:"))
    if ch == 1:
        data = int(input("Enter data:"))
        s.insert_b(data)
    elif ch == 2:
        data = int(input("Enter data:"))
        s.insert_e(data)
    elif ch == 3:
        data = int(input("Enter data:"))
        index = int(input("Enter index:"))
        s.insert_a(data, index)
    elif ch == 4:
        print(s.count())
    elif ch == 5:
        s.display()
    elif ch == 6:
        s.delete_b()
    elif ch == 7:
        s.delete_e()
    elif ch == 8:
        index = int(input("Enter index:"))
        s.delete_a(index)
    elif ch == 9:
        break
```

Output:

```
PS D:\DSA programs>  & 'C:\Users\BHARATH CHAND\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52242' '--' 'D:\DSA programs\dll.py'
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
7.Delete at end
8.Delete at any index
9.Exit
Enter your choice:1
Enter data:10
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
7.Delete at end
8.Delete at any index
9.Exit
Enter your choice:2
Enter data:20
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
```

```
PS D:\DSA programs>  & 'C:\Users\BHARATH CHAND\AppData\Local\Programs\Python\P
ython313\python.exe' 'c:\Users\BHARATH CHAND\.vscode\extensions\ms-python.debu
gpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52242' '--' 'D:\DSA pro
grams\dll.py'
7.Delete at end
8.Delete at any index
9.Exit
Enter your choice:3
Enter data:30
Enter index:1
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
7.Delete at end
8.Delete at any index
9.Exit
Enter your choice:4
3
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
7.Delete at end
8.Delete at any index
9.Exit
Enter your choice:5
10 30 20
```

```
PS D:\DSA programs>  & 'C:\Users\BHARATH CHAND\AppData\Local\Pr
ython313\python.exe' 'c:\Users\BHARATH CHAND\.vscode\extensions
gpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52242' '
grams\dll.py'
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
7.Delete at end
8.Delete at any index
9.Exit
Enter your choice:6
Deleted
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
7.Delete at end
8.Delete at any index
9.Exit
Enter your choice:7
Deleted
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
```

```
7.Delete at end
8.Delete at any index
9.Exit
Enter your choice:8
Enter index:0
Deleted
1.Insert at beginning
2.Insert at end
3.Insert at any index
4.Count
5.Display
6.Delete at beginning
7.Delete at end
8.Delete at any index
9.Exit
Enter your choice:9
PS D:\DSA programs>
```

WEEK-7

AIM: 7(A). Write a python program to implement stack operations such as push, pop, peek, display using an array.

CODE:



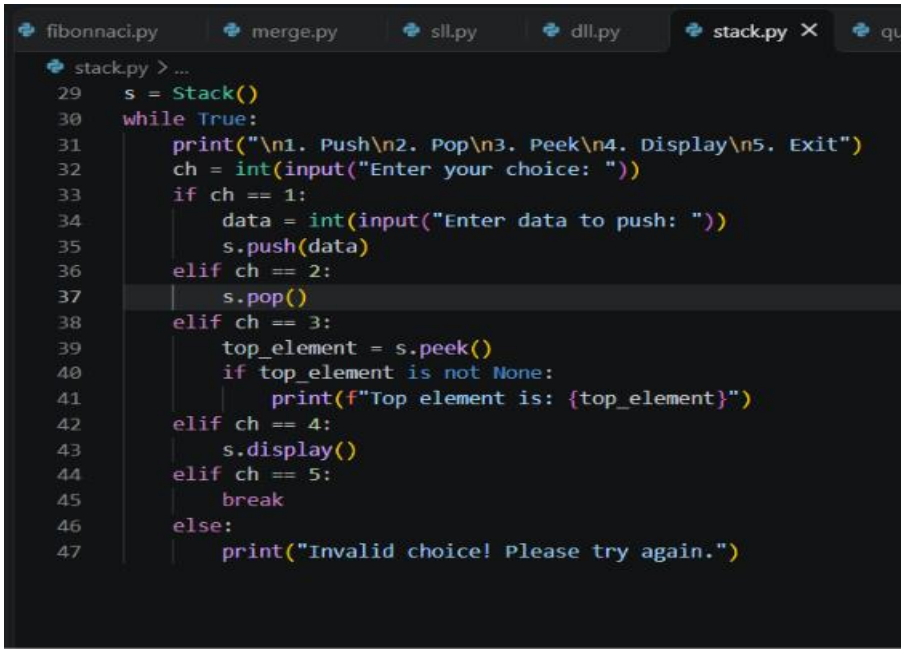
```python
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, data):
        self.stack.append(data)
        print(f"Pushed {data} into the stack.")

    def pop(self):
        if not self.stack:
            print("Stack Underflow! The stack is empty.")
            return None
        removed = self.stack.pop()
        print(f"Popped element: {removed}")
        return removed

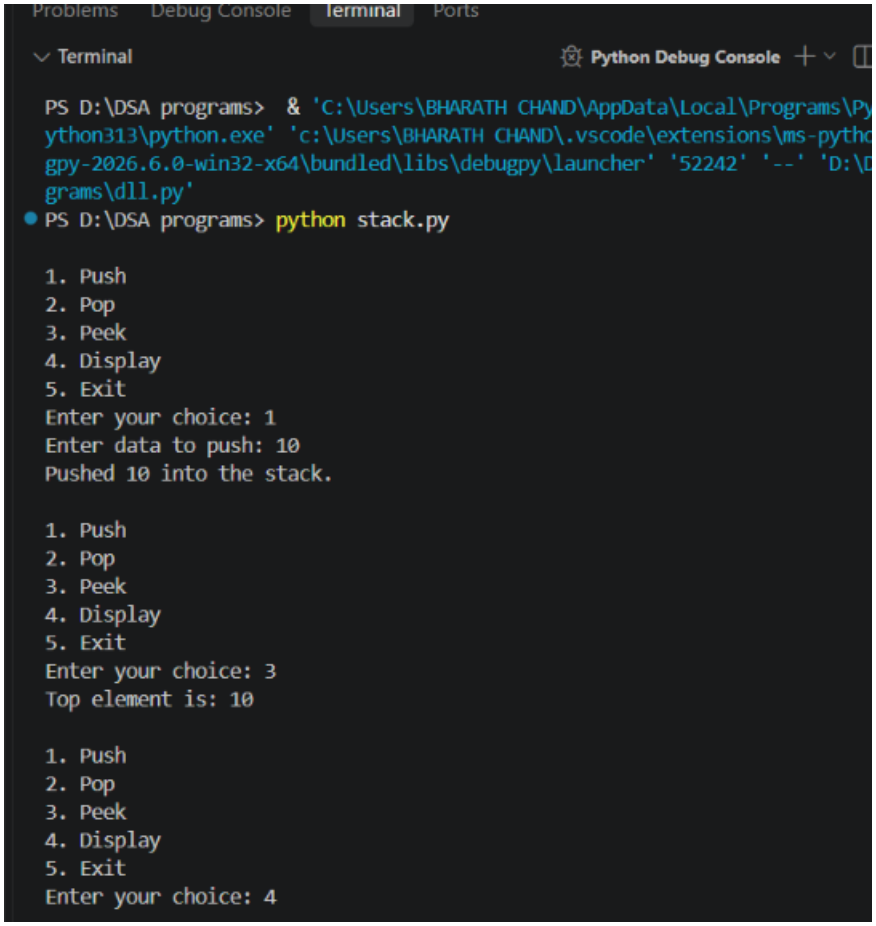
    def peek(self):
        if not self.stack:
            print("The stack is empty.")
            return None
        return self.stack[-1]

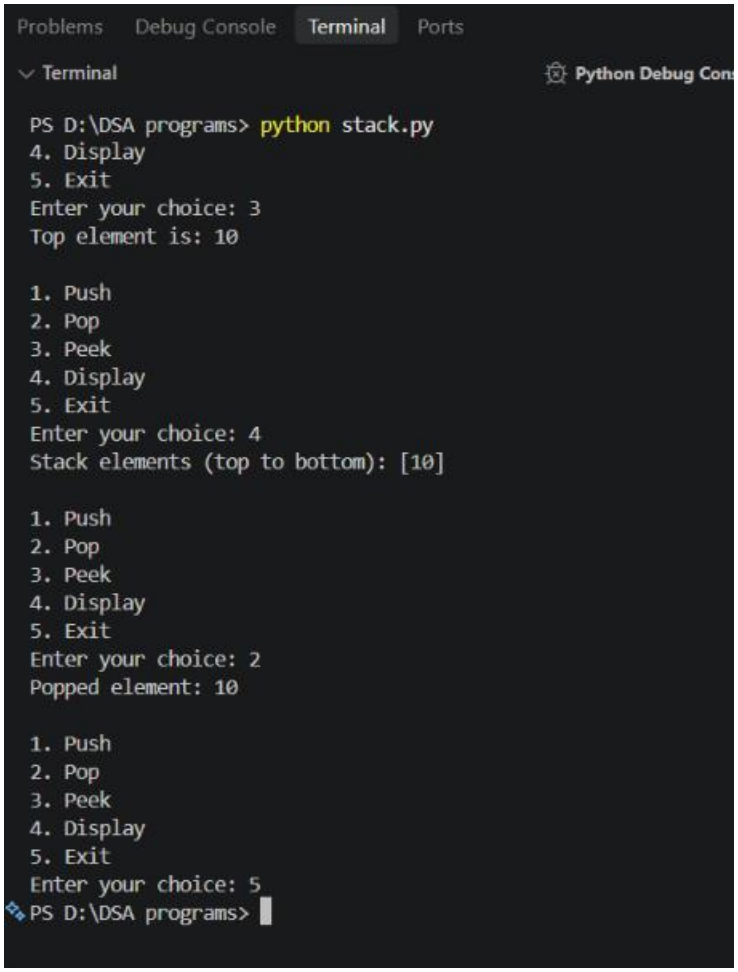
    def display(self):
        if not self.stack:
            print("The stack is empty.")
        else:
            print("Stack elements (top to bottom):", self.stack[::-1])
```



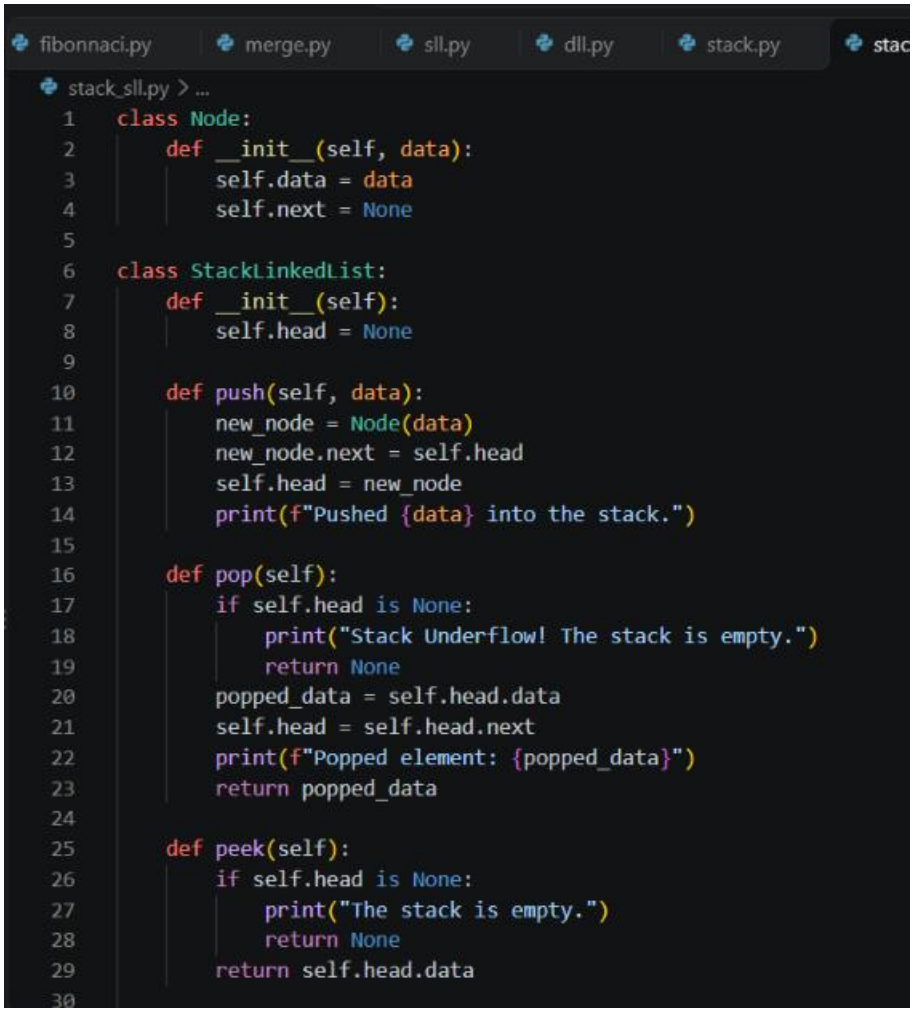
```python
s = Stack()
while True:
    print("\n1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        data = int(input("Enter data to push: "))
        s.push(data)
    elif ch == 2:
        s.pop()
    elif ch == 3:
        top_element = s.peek()
        if top_element is not None:
            print(f"Top element is: {top_element}")
    elif ch == 4:
        s.display()
    elif ch == 5:
        break
    else:
        print("Invalid choice! Please try again.")
```

Output:


Problems
Debug Console
Terminal
Ports
Terminal
Python Debug Console
PS D:\DSA programs> & 'C:\Users\BHARATH CHAND\AppData\Local\Programs\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52242' '--' 'D:\DSA programs\dll.py'
PS D:\DSA programs> python stack.py
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to push: 10
Pushed 10 into the stack.
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element is: 10
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4



7(B). Write a python program to implement stack operations such as push, pop, peek, display using a linked list

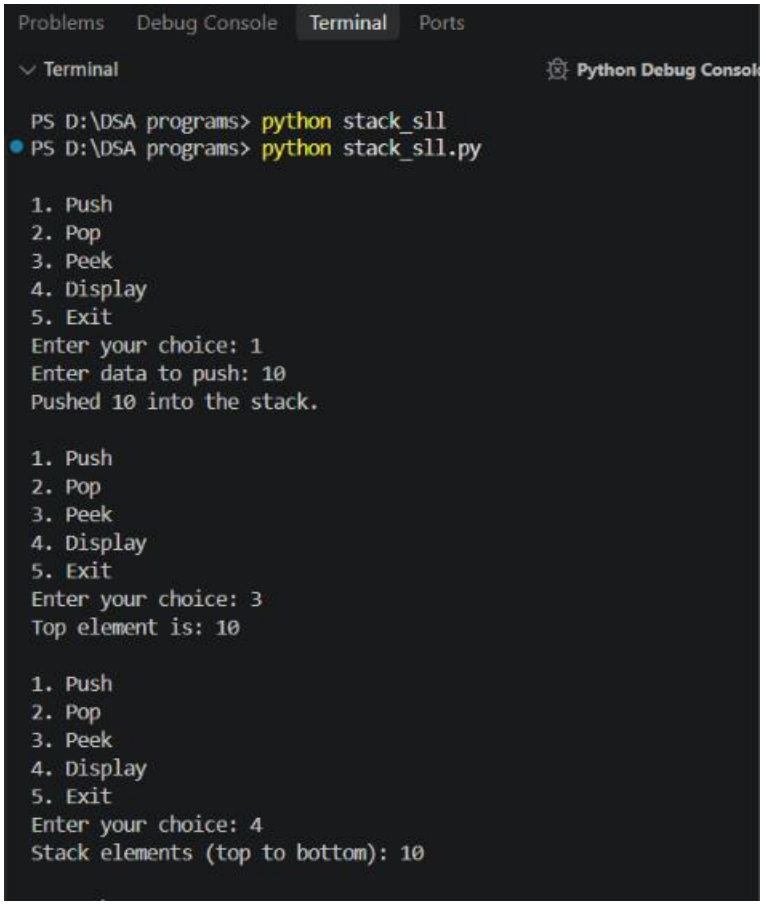

fibonnaci.py
merge.py
sll.py
dll.py
stack.py
stac
stack_sll.py > ...
class Node:
def __init__(self, data):
self.data = data
self.next = None
class StackLinkedList:
def __init__(self):
self.head = None
def push(self, data):
new_node = Node(data)
new_node.next = self.head
self.head = new_node
print(f"Pushed {data} into the stack.")
def pop(self):
if self.head is None:
print("Stack Underflow! The stack is empty.")
return None
popped_data = self.head.data
self.head = self.head.next
print(f"Popped element: {popped_data}")
return popped_data
def peek(self):
if self.head is None:
print("The stack is empty.")
return None
return self.head.data

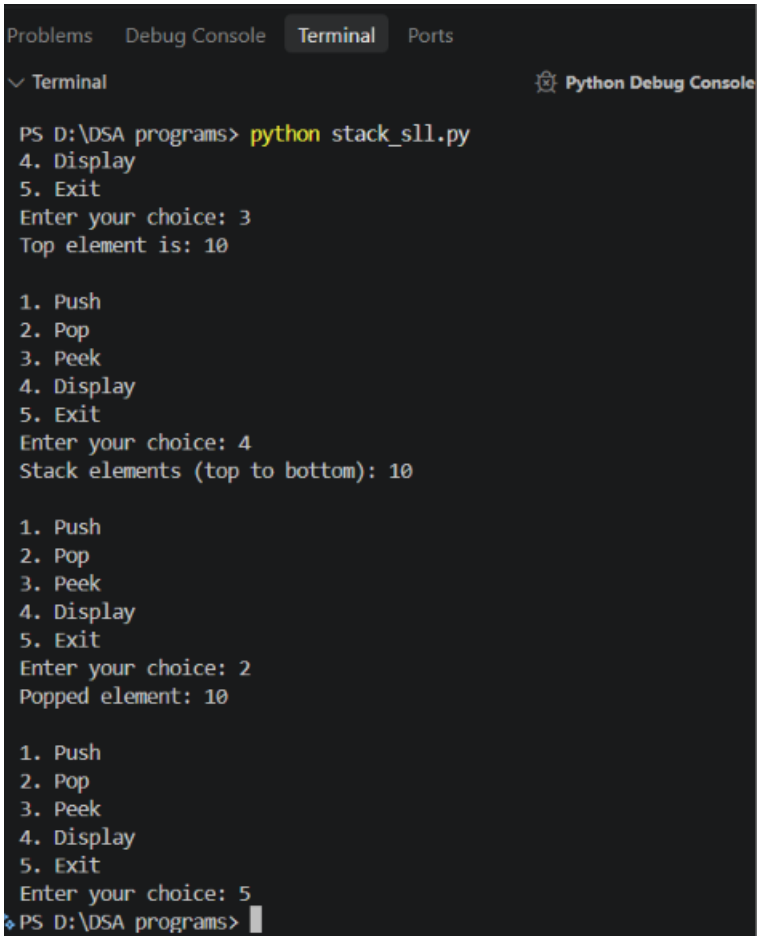
fibonnaci.py | merge.py | sll.py | dll.py | stack.py | stack_sll.py

stack_sll.py > ...

```
6   class StackLinkedList:
31      def display(self):
32          if self.head is None:
33              print("The stack is empty.")
34          else:
35              print("Stack elements (top to bottom):", end=" ")
36              temp = self.head
37              while temp:
38                  print(temp.data, end=" ")
39                  temp = temp.next
40              print()
41
42  s = StackLinkedList()
43  while True:
44      print("\n1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit")
45      ch = int(input("Enter your choice: "))
46      if ch == 1:
47          data = int(input("Enter data to push: "))
48          s.push(data)
49      elif ch == 2:
50          s.pop()
51      elif ch == 3:
52          top_element = s.peek()
53          if top_element is not None:
54              print(f"Top element is: {top_element}")
55      elif ch == 4:
56          s.display()
57      elif ch == 5:
58          break
59      else:
```

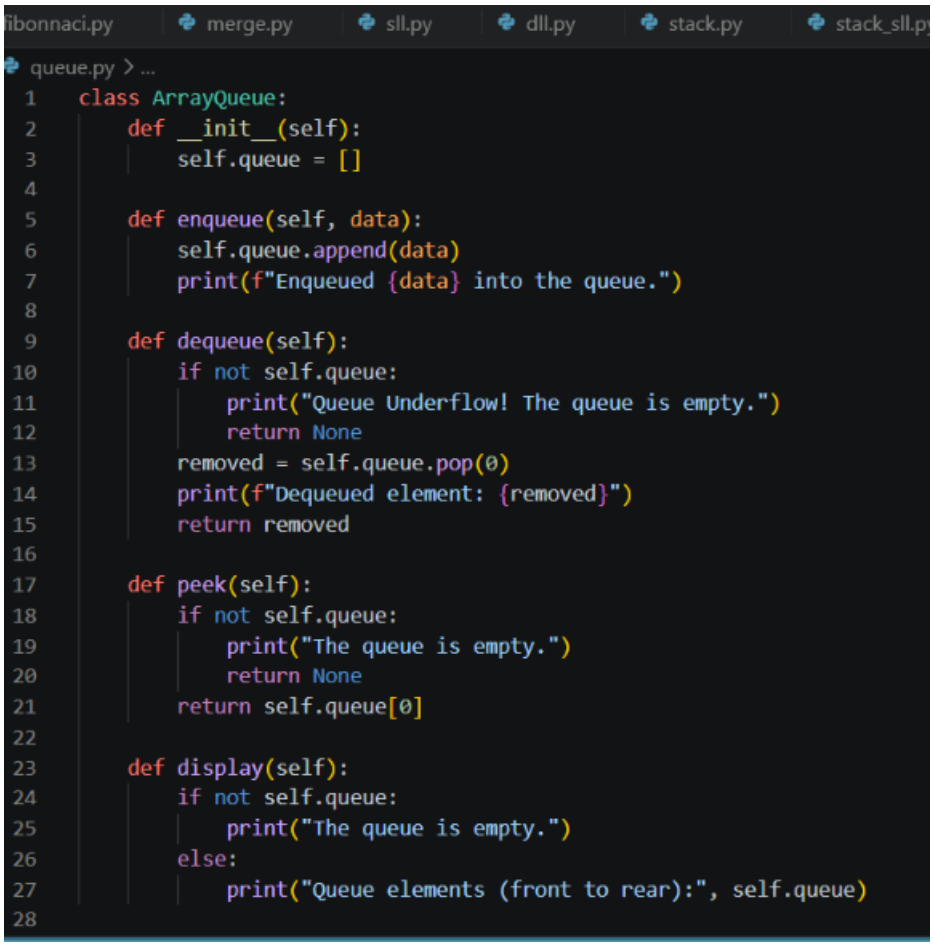
Output:


Problems
Debug Console
Terminal
Ports
Terminal
Python Debug Consol
PS D:\DSA programs> python stack_sll
PS D:\DSA programs> python stack_sll.py
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to push: 10
Pushed 10 into the stack.
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element is: 10
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (top to bottom): 10


Problems
Debug Console
Terminal
Ports
Terminal
Python Debug Console
PS D:\DSA programs> python stack_sll.py
4. Display
5. Exit
Enter your choice: 3
Top element is: 10
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (top to bottom): 10
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 10
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
PS D:\DSA programs>

WEEK-8

AIM: 8(A). Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.




```python
class ArrayQueue:
    def __init__(self):
        self.queue = []

    def enqueue(self, data):
        self.queue.append(data)
        print(f"Enqueued {data} into the queue.")

    def dequeue(self):
        if not self.queue:
            print("Queue Underflow! The queue is empty.")
            return None
        removed = self.queue.pop(0)
        print(f"Dequeued element: {removed}")
        return removed

    def peek(self):
        if not self.queue:
            print("The queue is empty.")
            return None
        return self.queue[0]

    def display(self):
        if not self.queue:
            print("The queue is empty.")
        else:
            print("Queue elements (front to rear):", self.queue)

```

fibonnaci.py | merge.py | sll.py | dll.py | stack.py | stack_sll

queue.py > ...

```python

q_arr = ArrayQueue()
while True:
    print("\n--- Array Queue Menu ---")
    print("1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        data = int(input("Enter data to enqueue: "))
        q_arr.enqueue(data)
    elif ch == 2:
        q_arr.dequeue()
    elif ch == 3:
        top_element = q_arr.peek()
        if top_element is not None:
            print(f"Front element is: {top_element}")
    elif ch == 4:
        q_arr.display()
    elif ch == 5:
        break
    else:
        print("Invalid choice! Please try again.")
```

Output:

Problems | Debug Console | Terminal | Ports

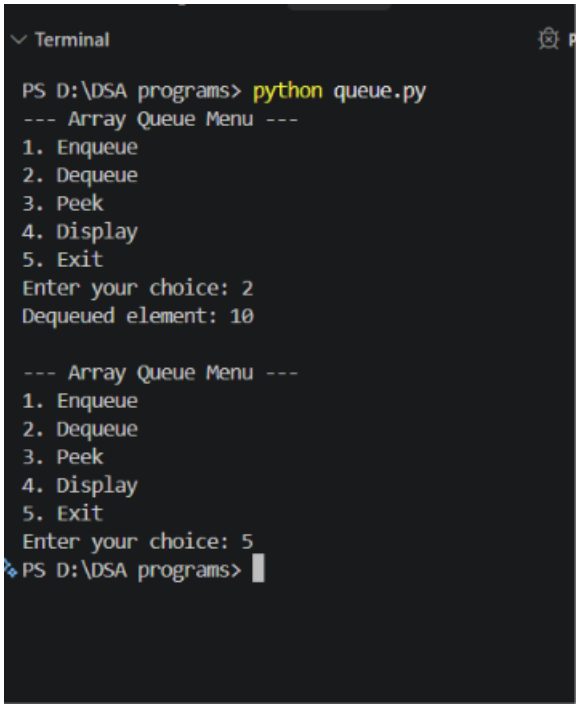
Terminal — Python Debug Console

```
PS D:\DSA programs> python stack_sll.py
PS D:\DSA programs> python queue.py

--- Array Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to enqueue: 10
Enqueued 10 into the queue.

--- Array Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element is: 10

--- Array Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements (front to rear): [10]
```



Code:

fibonnaci.py | merge.py | sll.py | dll.py | stack.py | stack

queue_sll.py > Node

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedListQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new_node = Node(data)
        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node
        print(f"Enqueued {data} into the queue.")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow! The queue is empty.")
            return None
        dequeued_data = self.front.data
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        print(f"Dequeued element: {dequeued_data}")
        return dequeued_data
```

fibonnaci.py | merge.py | sll.py | dll.py | stack.py | stack_sll.py

queue_sll.py > Node

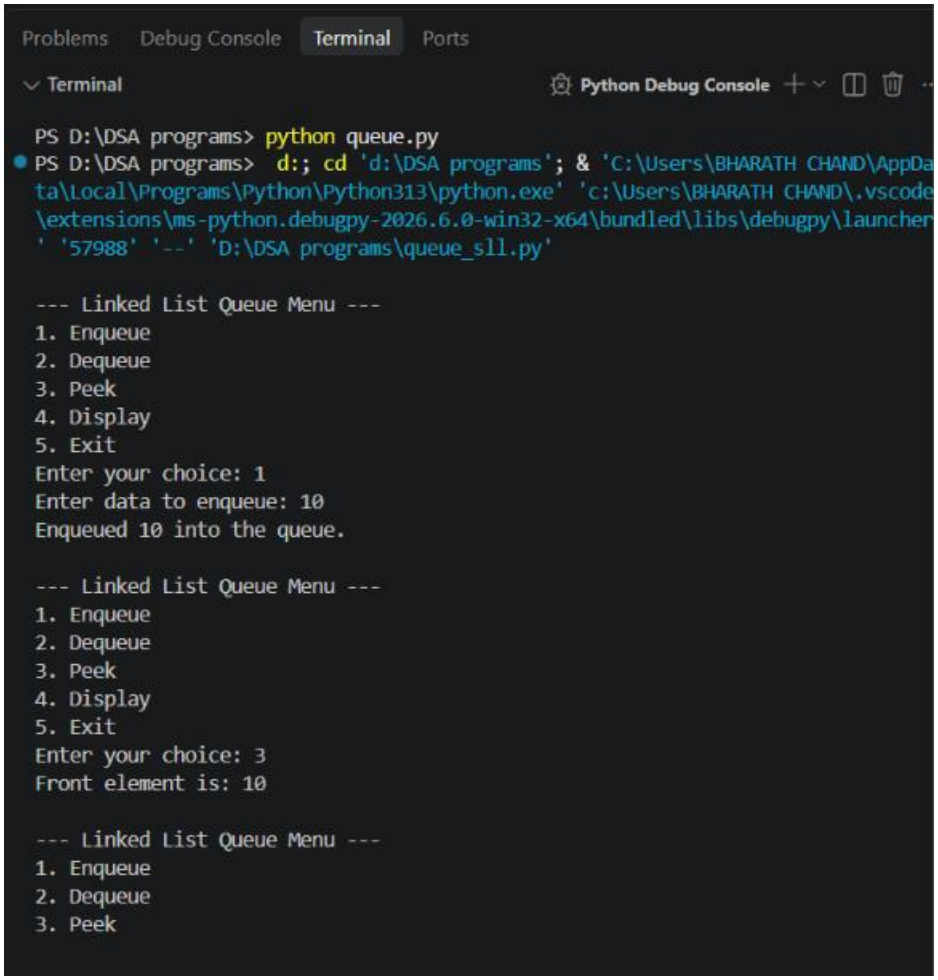
```
6   class LinkedListQueue:
31      def peek(self):
32          if self.front is None:
33              print("The queue is empty.")
34              return None
35          return self.front.data
36
37      def display(self):
38          if self.front is None:
39              print("The queue is empty.")
40          else:
41              print("Queue elements (front to rear):", end=" ")
42              temp = self.front
43              while temp:
44                  print(temp.data, end=" ")
45                  temp = temp.next
46              print()
47
48  q_ll = LinkedListQueue()
49  while True:
50      print("\n--- Linked List Queue Menu ---")
51      print("1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. Exit")
52      ch = int(input("Enter your choice: "))
53      if ch == 1:
54          data = int(input("Enter data to enqueue: "))
55          q_ll.enqueue(data)
56      elif ch == 2:
57          q_ll.dequeue()
58      elif ch == 3:
```

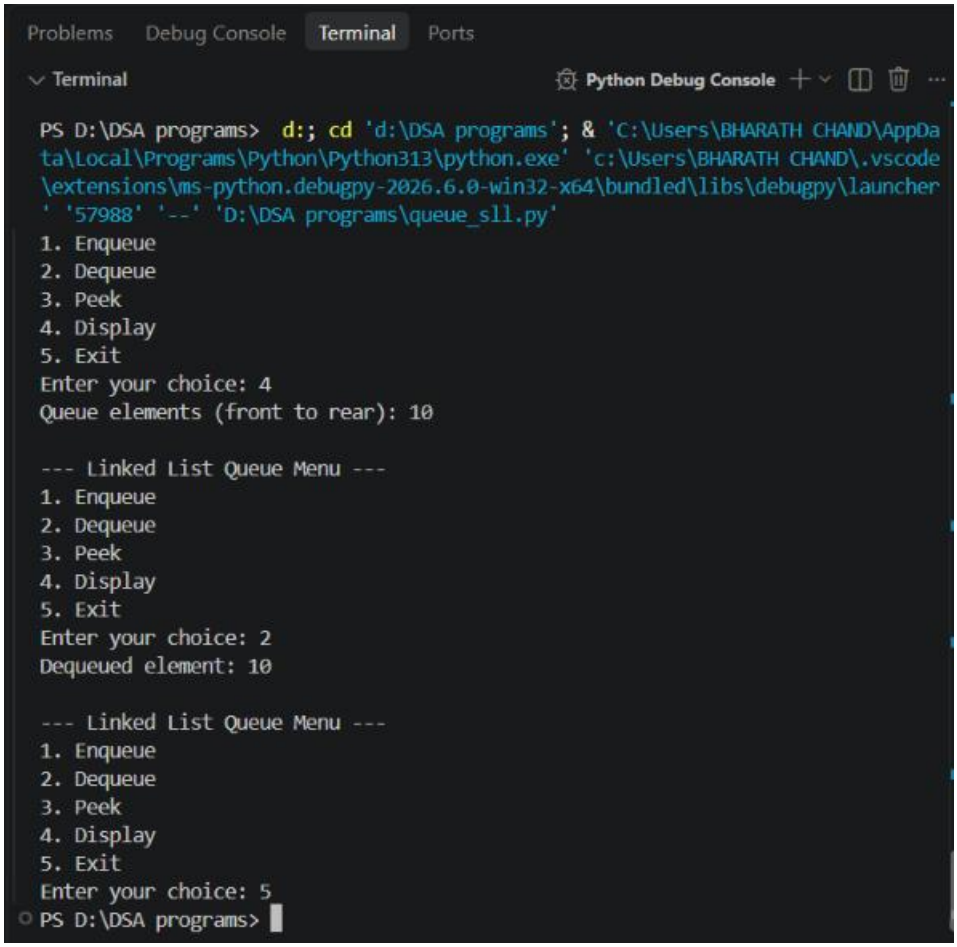
fibonnaci.py | merge.py | sll.py | dll.py | stack.py

queue_sll.py > Node

```
59              top_element = q_ll.peek()
60              if top_element is not None:
61                  print(f"Front element is: {top_element}")
62          elif ch == 4:
63              q_ll.display()
64          elif ch == 5:
65              break
66          else:
67              print("Invalid choice! Please try again.")
```

Output:


Problems
Debug Console
Terminal
Ports
Terminal
Python Debug Console
PS D:\DSA programs> python queue.py
PS D:\DSA programs> d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHAND\AppDa
ta\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscode
\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher
' '57988' '--' 'D:\DSA programs\queue_sll.py'
--- Linked List Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to enqueue: 10
Enqueued 10 into the queue.
--- Linked List Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element is: 10
--- Linked List Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek


Problems
Debug Console
Terminal
Ports
Terminal
Python Debug Console
PS D:\DSA programs> d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHAND\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '57988' '--' 'D:\DSA programs\queue_sll.py'
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements (front to rear): 10
--- Linked List Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Dequeued element: 10
--- Linked List Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
PS D:\DSA programs>

8(B). Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

fibonnaci.py | merge.py | sll.py | dll.py | stack.py | stack_sll.py

```python
class ArrayCircularQueue:
    def __init__(self, capacity):
        self.capacity = capacity
        self.queue = [None] * capacity
        self.front = -1
        self.rear = -1
        self.size = 0

    def enqueue(self, data):
        if self.size == self.capacity:
            print("Queue Overflow! The circular queue is full.")
            return
        if self.front == -1:
            self.front = 0
        self.rear = (self.rear + 1) % self.capacity
        self.queue[self.rear] = data
        self.size += 1
        print(f"Enqueued {data} into the circular queue.")

    def dequeue(self):
        if self.size == 0:
            print("Queue Underflow! The circular queue is empty.")
            return None
        removed = self.queue[self.front]
        self.queue[self.front] = None
        if self.front == self.rear:
            self.front = self.rear = -1
        else:
            self.front = (self.front + 1) % self.capacity
        self.size -= 1
        print(f"Dequeued element: {removed}")
        return removed
```

ibonnaci.py | merge.py | sll.py | dll.py | stack.py | stack

```python
class ArrayCircularQueue:
    def peek(self):
        if self.size == 0:
            print("The circular queue is empty.")
            return None
        return self.queue[self.front]

    def display(self):
        if self.size == 0:
            print("The circular queue is empty.")
        else:
            print("Queue elements (front to rear):", end=" ")
            i = self.front
            for _ in range(self.size):
                print(self.queue[i], end=" ")
                i = (i + 1) % self.capacity
            print()
```

fibonnaci.py | merge.py | sll.py | dll.py | stack.py | stack_sll.p

```python
cq_arr = ArrayCircularQueue(cap)
while True:
    print("\n--- Array Circular Queue Menu ---")
    print("1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        data = int(input("Enter data to enqueue: "))
        cq_arr.enqueue(data)
    elif ch == 2:
        cq_arr.dequeue()
    elif ch == 3:
        top_element = cq_arr.peek()
        if top_element is not None:
            print(f"Front element is: {top_element}")
    elif ch == 4:
        cq_arr.display()
    elif ch == 5:
        break
    else:
        print("Invalid choice! Please try again.")
```

Output:

Problems | Debug Console | Terminal | Ports

Terminal — Python Debug Console

```
PS D:\DSA programs> python c_queue.py

--- Array Circular Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to enqueue: 10
Enqueued 10 into the circular queue.

--- Array Circular Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element is: 10

--- Array Circular Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements (front to rear): 10
```


Terminal
PS D:\DSA programs> python c_queue.py
Queue elements (front to rear): 10
--- Array Circular Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Dequeued element: 10
--- Array Circular Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
PS D:\DSA programs>

Code:

bonnaci.py | merge.py | sll.py | dll.py | stack.py | stack_sll.py | queue.py

c_queue_sll.py > ...

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedListCircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new_node = Node(data)
        if self.front is None:
            self.front = new_node
            self.rear = new_node
            self.rear.next = self.front
        else:
            self.rear.next = new_node
            self.rear = new_node
            self.rear.next = self.front
        print(f"Enqueued {data} into the circular queue.")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow! The circular queue is empty.")
            return None
        dequeued_data = self.front.data
        if self.front == self.rear:
            self.front = None
            self.rear = None
        else:
            self.front = self.front.next
            self.rear.next = self.front
        print(f"Dequeued element: {dequeued_data}")
        return dequeued_data
```

```
36
37      def peek(self):
38          if self.front is None:
39              print("The circular queue is empty.")
40              return None
41          return self.front.data
42
43      def display(self):
44          if self.front is None:
45              print("The circular queue is empty.")
46          else:
47              print("Queue elements (front to rear):", end=" ")
48              temp = self.front
49              while True:
50                  print(temp.data, end=" ")
51                  temp = temp.next
52                  if temp == self.front:
53                      break
54              print()
55
```

```
cq_ll = LinkedListCircularQueue()
while True:
    print("\n--- Linked List Circular Queue Menu ---")
    print("1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        data = int(input("Enter data to enqueue: "))
        cq_ll.enqueue(data)
    elif ch == 2:
        cq_ll.dequeue()
    elif ch == 3:
        top_element = cq_ll.peek()
        if top_element is not None:
            print(f"Front element is: {top_element}")
    elif ch == 4:
        cq_ll.display()
    elif ch == 5:
        break
    else:
        print("Invalid choice! Please try again.")
```

Output:

```
PS D:\DSA programs>  d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHAND\AppDa
ta\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscode
\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher
' '59666' '--' 'D:\DSA programs\c_queue_sll.py'

--- Linked List Circular Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to enqueue: 10
Enqueued 10 into the circular queue.

--- Linked List Circular Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element is: 10

--- Linked List Circular Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
```

```
Problems  Debug Console  Terminal  Ports

Terminal                                Python Debug Console

PS D:\DSA programs>  d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHAND\AppDa
ta\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscode
\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher
' '59666' '--' 'D:\DSA programs\c_queue_sll.py'
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements (front to rear): 10

--- Linked List Circular Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Dequeued element: 10

--- Linked List Circular Queue Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
PS D:\DSA programs>
```

WEEK-9:

AIM: Write a python program to implement a Binary Search Tree (BST) and perform the following operations.

1. Insertion

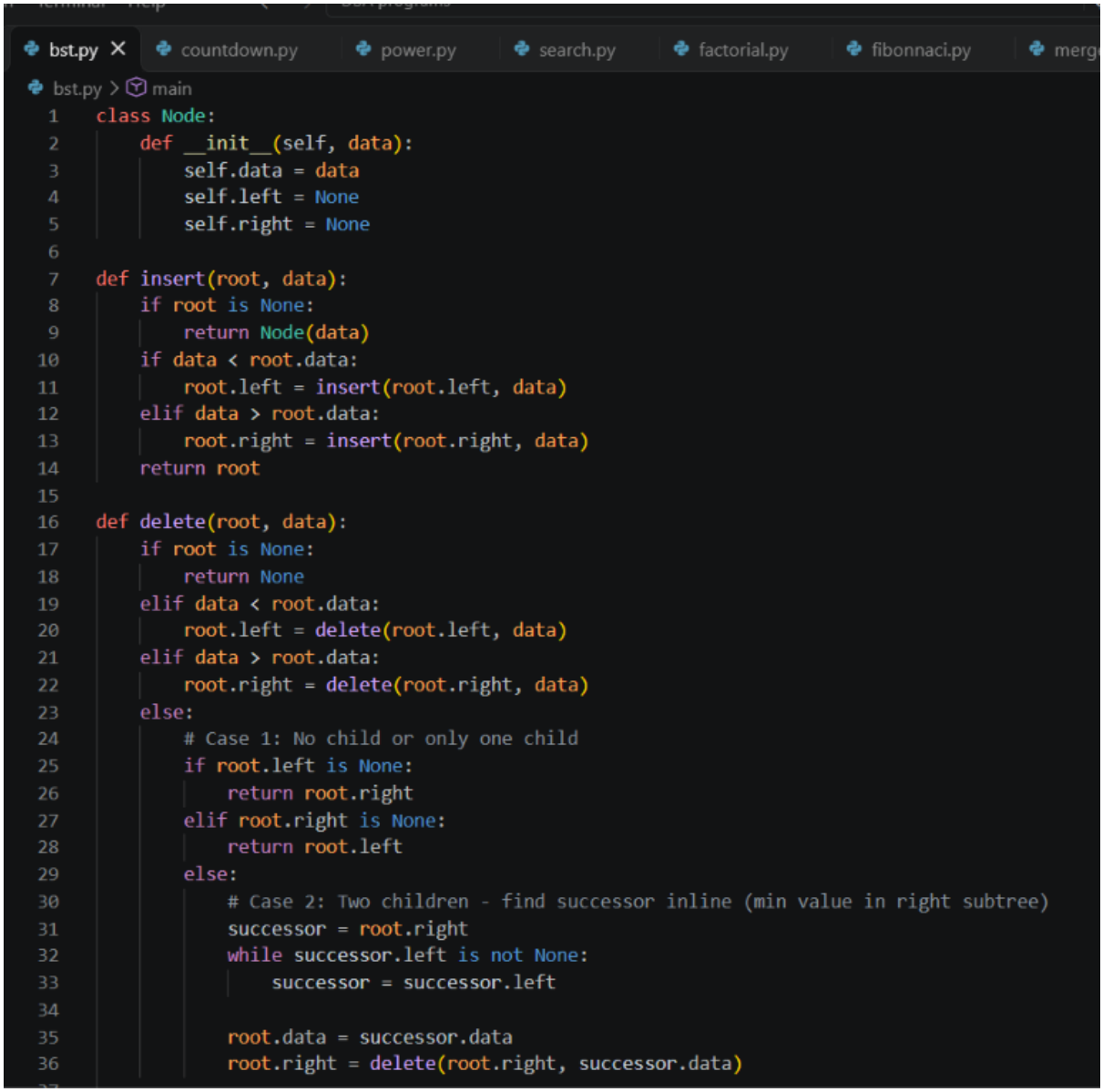
2. Deletion

3. Tree Traversals – in-order, pre-order and post-order

4. Count Leaf Nodes

5. Count Internal Nodes

6. Height of the Tree



bst.py | countdown.py | power.py | search.py | factorial.py | fibonnaci.py

bst.py > main

```
16  def delete(root, data):
37
38      return root
39
40  def preorder(root):
41      if root is not None:
42          print(root.data, end=" ")
43          preorder(root.left)
44          preorder(root.right)
45
46  def inorder(root):
47      if root is not None:
48          inorder(root.left)
49          print(root.data, end=" ")
50          inorder(root.right)
51
52  def postorder(root):
53      if root is not None:
54          postorder(root.left)
55          postorder(root.right)
56          print(root.data, end=" ")
57
58  def count_leaf_nodes(root):
59      if root is None:
60          return 0
61      if root.left is None and root.right is None:
62          return 1
63      return count_leaf_nodes(root.left) + count_leaf_nodes(root.right)
64
65  def count_internal_nodes(root):
66      if root is None:
67          return 0
68      if root.left is None and root.right is None:
69          return 0
70      return 1 + count_internal_nodes(root.left) + count_internal_nodes(root.right)
71
```

bst.py | countdown.py | power.py | search.py | factorial.py

bst.py > main

```
73  def main():
74      root = None
75      while True:
76          print("\n=========================")
77          print("      BST TREE MENU        ")
78          print("=========================")
79          print("1. Insert Node")
80          print("2. Delete Node")
81          print("3. Preorder Traversal")
82          print("4. Inorder Traversal")
83          print("5. Postorder Traversal")
84          print("6. Count Leaf Nodes")
85          print("7. Count Internal Nodes")
86          print("8. Exit")
87
88          choice = input("Enter your choice (1-8): ").strip()
89
90          if choice == '1':
91              val = int(input("Enter value to insert: "))
92              root = insert(root, val)
93              print(f"Successfully inserted {val}.")
94
95          elif choice == '2':
96              val = int(input("Enter value to delete: "))
97              root = delete(root, val)
98              print(f"Successfully deleted {val} (if present).")
99
100         elif choice == '3':
101             print("Preorder Traversal: ", end="")
102             preorder(root)
103             print()
104
105         elif choice == '4':
106             print("Inorder Traversal: ", end="")
107             inorder(root)
108             print()
```

bst.py | countdown.py | power.py | search.py | factorial.py | fibonnaci.py

bst.py > main

```
73   def main():
110          elif choice == '5':
111              print("Postorder Traversal: ", end="")
112              postorder(root)
113              print()
114
115          elif choice == '6':
116              print(f"Number of leaf nodes: {count_leaf_nodes(root)}")
117
118          elif choice == '7':
119              print(f"Number of internal nodes: {count_internal_nodes(root)}")
120
121          elif choice == '8':
122              print("Exiting program. Goodbye!")
123              break
124          else:
125              print("Invalid choice! Please select an option between 1 and 8.")
126
127  if __name__ == "__main__":
128      main()
```

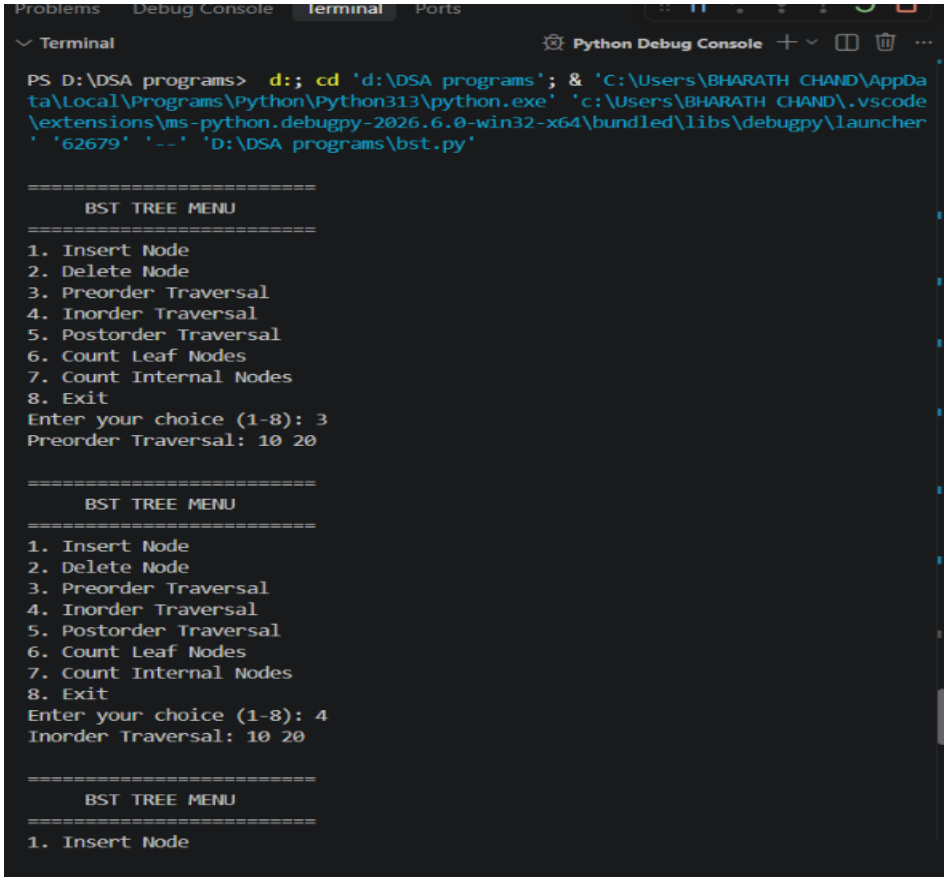
Output:

```
PS D:\DSA programs>  d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHAND\AppDa
ta\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscode
\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher
' '62679' '--' 'D:\DSA programs\bst.py'

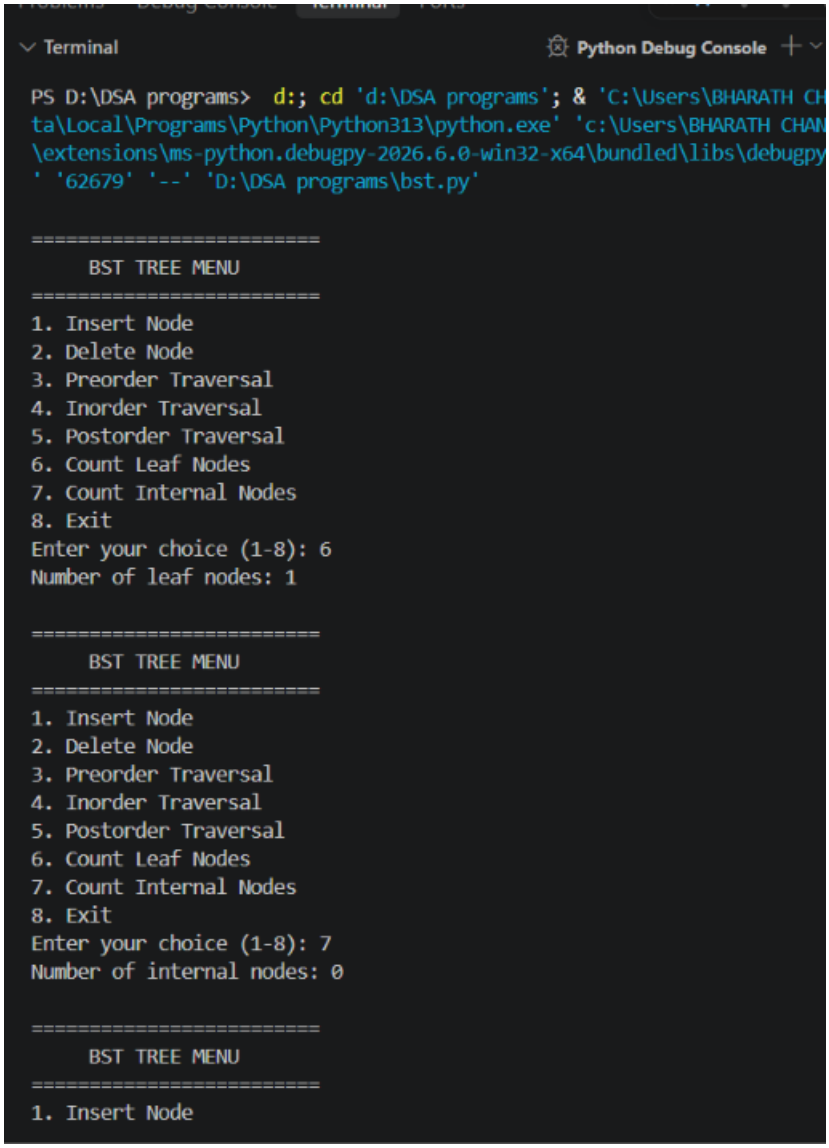
========================
     BST TREE MENU
========================
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 1
Enter value to insert: 10
Successfully inserted 10.

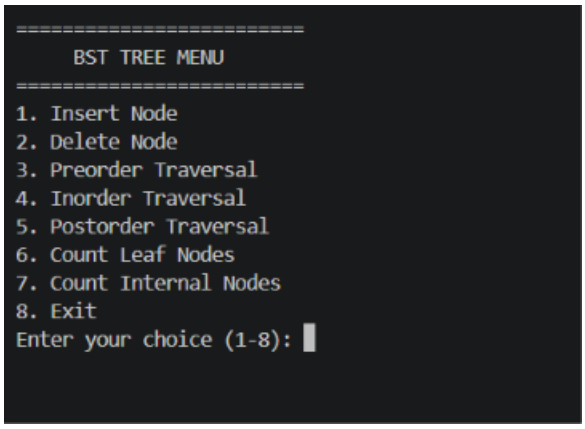
========================
     BST TREE MENU
========================
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 1
Enter value to insert: 20
Successfully inserted 20.

========================
     BST TREE MENU
```


Problems
Debug Console
Terminal
Ports
Terminal
Python Debug Console
PS D:\DSA programs> d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHAND\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '62679' '--' 'D:\DSA programs\bst.py'
BST TREE MENU
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 3
Preorder Traversal: 10 20
BST TREE MENU
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 4
Inorder Traversal: 10 20
BST TREE MENU
1. Insert Node


Problems
Debug Console
Terminal
Ports
Terminal
Python Debug Console
PS D:\DSA programs> d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '62679' '--' 'D:\DSA programs\bst.py'
BST TREE MENU
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 5
Postorder Traversal: 20 10
BST TREE MENU
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 2
Enter value to delete: 10
Successfully deleted 10 (if present).
BST TREE MENU


Terminal
Python Debug Console
PS D:\DSA programs> d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CH
ta\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAN
\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy
' '62679' '--' 'D:\DSA programs\bst.py'
BST TREE MENU
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 6
Number of leaf nodes: 1
BST TREE MENU
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 7
Number of internal nodes: 0
BST TREE MENU
1. Insert Node


BST TREE MENU
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8):

WEEK-10

AIM: Write a menu driven python program to implement an AVL Tree and perform the following operations.

1. Insertion
2. Deletion
3. Tree Traversals
4. Count Leaf Nodes
5. Count Internal Nodes
6. Height of the Tree

avl.py bst.py countdown.py power.py search.py factorial.py fibonnaci.p

avl.py > ...

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.h = 0

def height(root):
    if root is None:
        return -1
    else:
        return root.h

def ll(x):
    y = x.left
    x.left = y.right
    y.right = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

def rr(x):
    y = x.right
    x.right = y.left
    y.left = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

def lr(x):
    x.left = rr(x.left)
    return ll(x)

def rl(x):
    x.right = ll(x.right)
    return rr(x)
```

avl.py | bst.py | countdown.py | power.py | search.py | factorial.py | fibonnaci.py

avl.py > ...

```python
def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)

def inorder(root):
    if root is not None:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)

def postorder(root):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

# --- New Functions for Counting Nodes ---
def count_leaf_nodes(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaf_nodes(root.left) + count_leaf_nodes(root.right)

def count_internal_nodes(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return 1 + count_internal_nodes(root.left) + count_internal_nodes(root.right)

def insert(root, data):
    if root is None:
        return Node(data)
```

avl.py | bst.py | countdown.py | power.py | search.py | factoria

avl.py > ...

```python
def delete(root, data):
    if root is None:
        return None
    if data < root.data:
        root.left = delete(root.left, data)
        if height(root.right) - height(root.left) == 2:
            if height(root.right.left) > height(root.right.right):
                root = rl(root)
            else:
                root = rr(root)
    elif data > root.data:
        root.right = delete(root.right, data)
        if height(root.left) - height(root.right) == 2:
            if height(root.left.right) > height(root.left.left):
                root = lr(root)
            else:
                root = ll(root)
    else:
        if root.left is None and root.right is None:
            return None
        elif root.right is None:
            return root.left
        elif root.left is None:
            return root.right
        else:
            successor = root.right
            while successor.left is not None:
                successor = successor.left
            root.data = successor.data
            root.right = delete(root.right, successor.data)

    if root is None:
        return None
    root.h = max(height(root.left), height(root.right)) + 1
    return root
```

avl.py bst.py countdown.py power.py search.py factorial.py

avl.py > ...

```python
def main():
    root = None
    while True:
        print("\n=========================")
        print("      AVL TREE MENU       ")
        print("=========================")
        print("1. Insert Node")
        print("2. Delete Node")
        print("3. Preorder Traversal")
        print("4. Inorder Traversal")
        print("5. Postorder Traversal")
        print("6. Count Leaf Nodes")
        print("7. Count Internal Nodes")
        print("8. Exit")

        choice = input("Enter your choice (1-8): ").strip()

        if choice == '1':
            val = int(input("Enter value to insert: "))
            root = insert(root, val)
            print(f"Successfully inserted {val}.")

        elif choice == '2':
            val = int(input("Enter value to delete: "))
            root = delete(root, val)
            print(f"Successfully deleted {val} (if present).")

        elif choice == '3':
            print("Preorder Traversal: ", end="")
            preorder(root)
            print()

        elif choice == '4':
            print("Inorder Traversal: ", end="")
            inorder(root)
            print()
```

avl.py bst.py countdown.py power.py search.py factorial.py fibonnac

avl.py > ...

```python
def main():
        elif choice == '5':
            print("Postorder Traversal: ", end="")
            postorder(root)
            print()

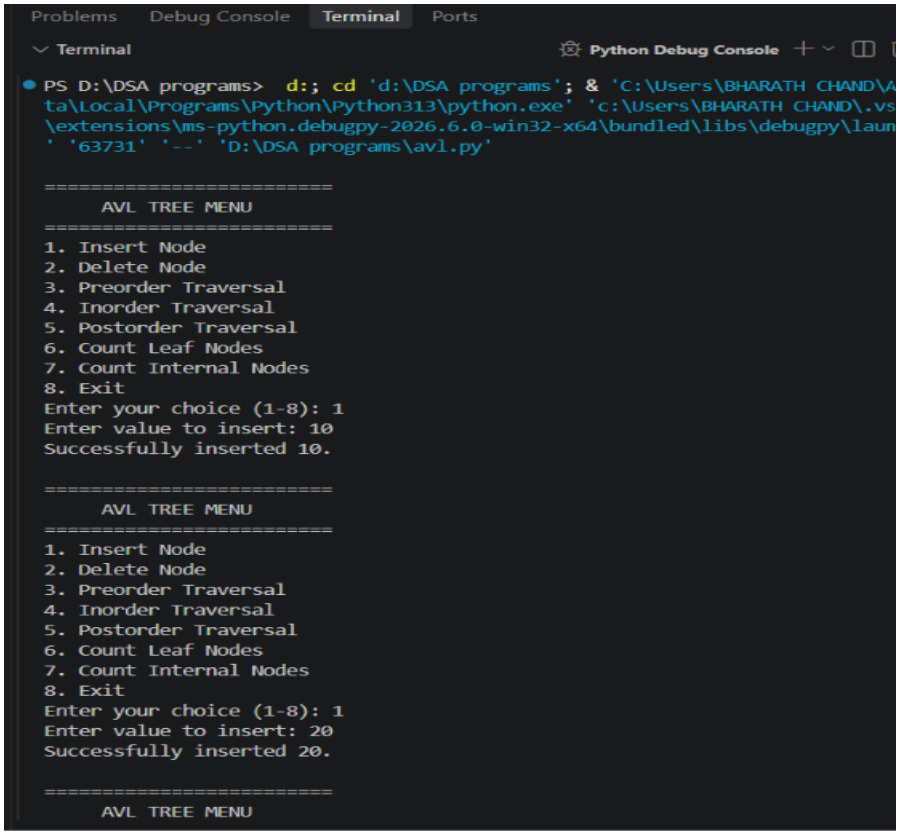
        elif choice == '6':
            print(f"Number of leaf nodes: {count_leaf_nodes(root)}")

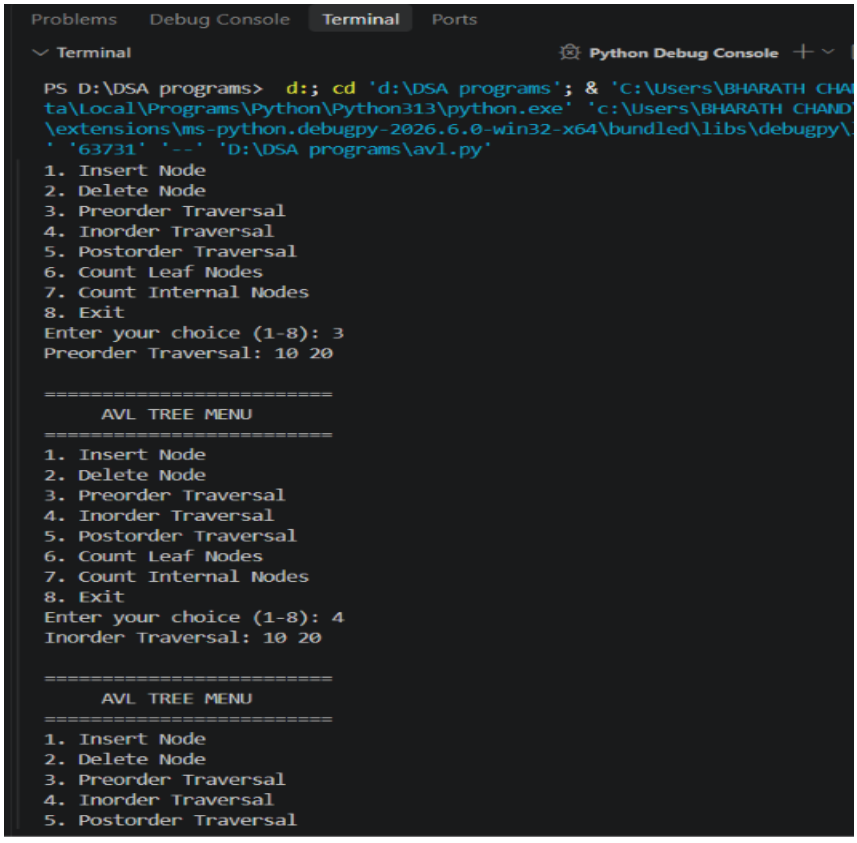
        elif choice == '7':
            print(f"Number of internal nodes: {count_internal_nodes(root)}")

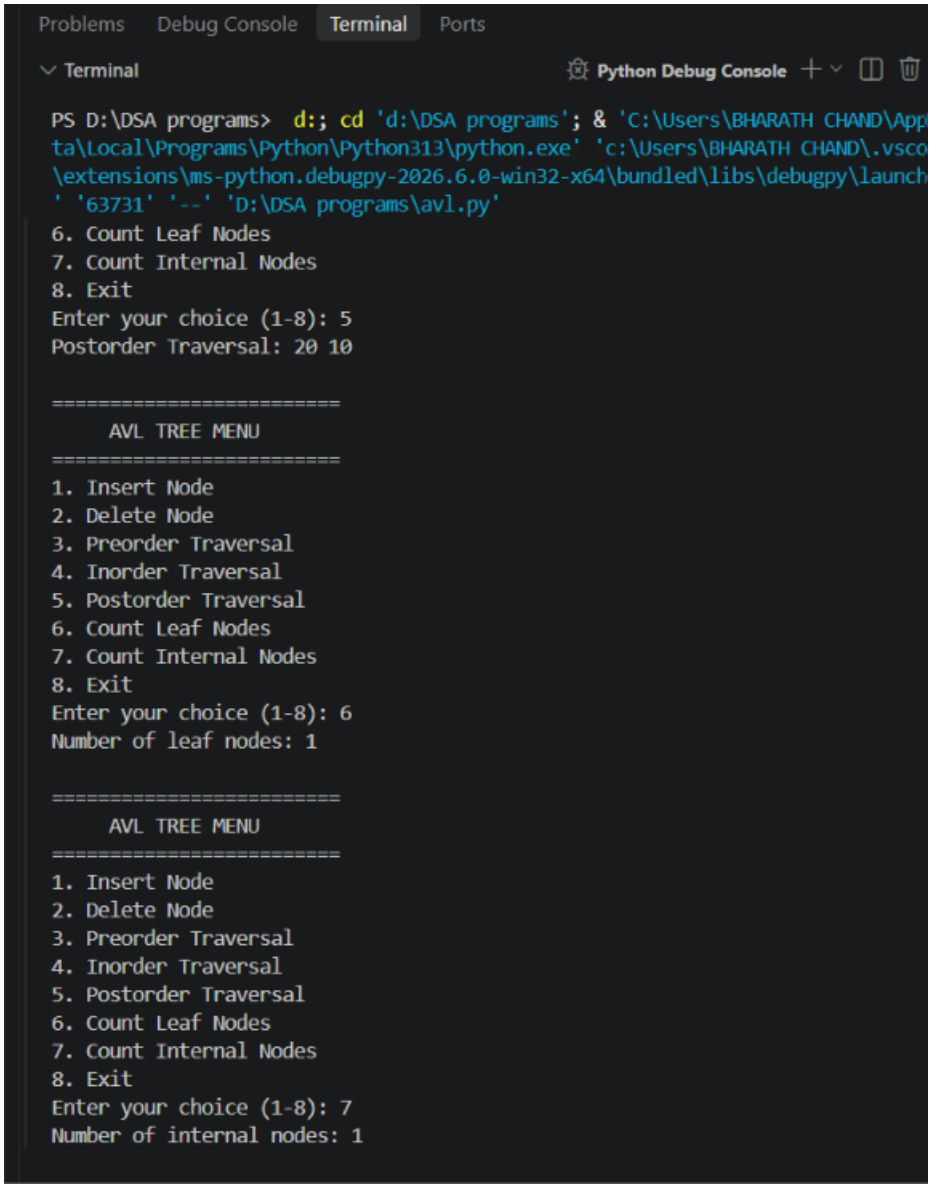
        elif choice == '8':
            print("Exiting program. Goodbye!")
            break
        else:
            print("Invalid choice! Please select an option between 1 and 8.")

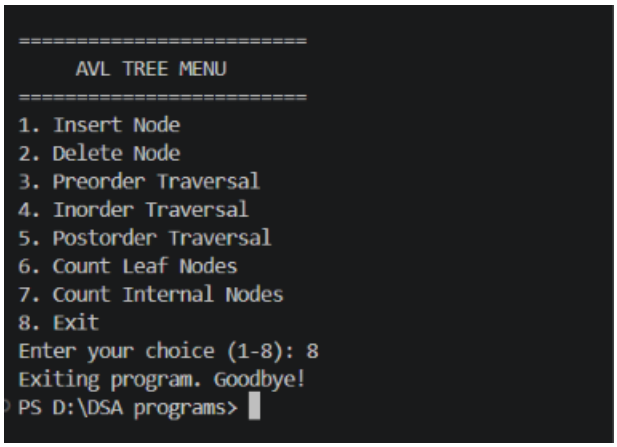
if __name__ == "__main__":
    main()
```

Output:


Problems
Debug Console
Terminal
Ports
Terminal
Python Debug Console
PS D:\DSA programs> d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHAND\A
ta\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vs
\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\laun
' '63731' '--' 'D:\DSA programs\avl.py'
AVL TREE MENU
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 1
Enter value to insert: 10
Successfully inserted 10.
AVL TREE MENU
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 1
Enter value to insert: 20
Successfully inserted 20.
AVL TREE MENU


Problems
Debug Console
Terminal
Ports
Terminal
Python Debug Console
PS D:\DSA programs> d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHA
ta\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND
\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\
' '63731' '--' 'D:\DSA programs\avl.py'
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 3
Preorder Traversal: 10 20
AVL TREE MENU
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 4
Inorder Traversal: 10 20
AVL TREE MENU
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal


Problems
Debug Console
Terminal
Ports
Terminal
Python Debug Console
PS D:\DSA programs> d:; cd 'd:\DSA programs'; & 'C:\Users\BHARATH CHAND\App
ta\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vsco
\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launch
' '63731' '--' 'D:\DSA programs\avl.py'
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 5
Postorder Traversal: 20 10
AVL TREE MENU
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 6
Number of leaf nodes: 1
AVL TREE MENU
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 7
Number of internal nodes: 1


AVL TREE MENU
1. Insert Node
2. Delete Node
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Exit
Enter your choice (1-8): 8
Exiting program. Goodbye!
PS D:\DSA programs>

WEEK-11:

AIM: Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations:

1. Insertion of an element into the Heap.

2. Deletion of the root element from the Heap.

3. Display the array representation of the Heap.

4. Exit the program.

avl.py | class BinaryHeap: Untitled-1 | bst.py | countdown.py | power.py | search.py

```python
class BinaryHeap:
    def __init__(self, heap_type="max"):
        self.heap = []
        self.heap_type = heap_type.lower()  # "max" or "min"

    def _is_valid_order(self, parent, child):
        if self.heap_type == "max":
            return self.heap[parent] >= self.heap[child]
        else:
            return self.heap[parent] <= self.heap[child]

    def insert(self, data):
        self.heap.append(data)
        self._heapify_up(len(self.heap) - 1)
        print(f"Inserted {data} into the {self.heap_type} heap.")

    def _heapify_up(self, index):
        parent = (index - 1) // 2
        while index > 0 and not self._is_valid_order(parent, index):
            self.heap[parent], self.heap[index] = self.heap[index], self.heap[parent]
            index = parent
            parent = (index - 1) // 2

    def delete_root(self):
        if not self.heap:
            print("Heap Underflow! The heap is empty.")
            return None

        root = self.heap[0]
        last_element = self.heap.pop()

        if self.heap:
            self.heap[0] = last_element
            self._heapify_down(0)

        print(f"Deleted root element: {root}")
        return root
```

avl.py | class BinaryHeap: Untitled-1 | bst.py | countdown.py | power.py | search.py

```
class BinaryHeap:
    def _heapify_down(self, index):
        length = len(self.heap)
        while True:
            left = 2 * index + 1
            right = 2 * index + 2
            target = index

            if left < length and not self._is_valid_order(target, left):
                target = left
            if right < length and not self._is_valid_order(target, right):
                target = right

            if target != index:
                self.heap[index], self.heap[target] = self.heap[target], self.heap[index]
                index = target
            else:
                break

    def display(self):
        if not self.heap:
            print("The heap is empty.")
        else:
            print(f"Heap array representation ({self.heap_type} heap):", self.heap)

print("Select Heap Type:")
print("1. Max Heap")
print("2. Min Heap")
choice = int(input("Enter choice (1 or 2): "))
h_type = "max" if choice == 1 else "min"

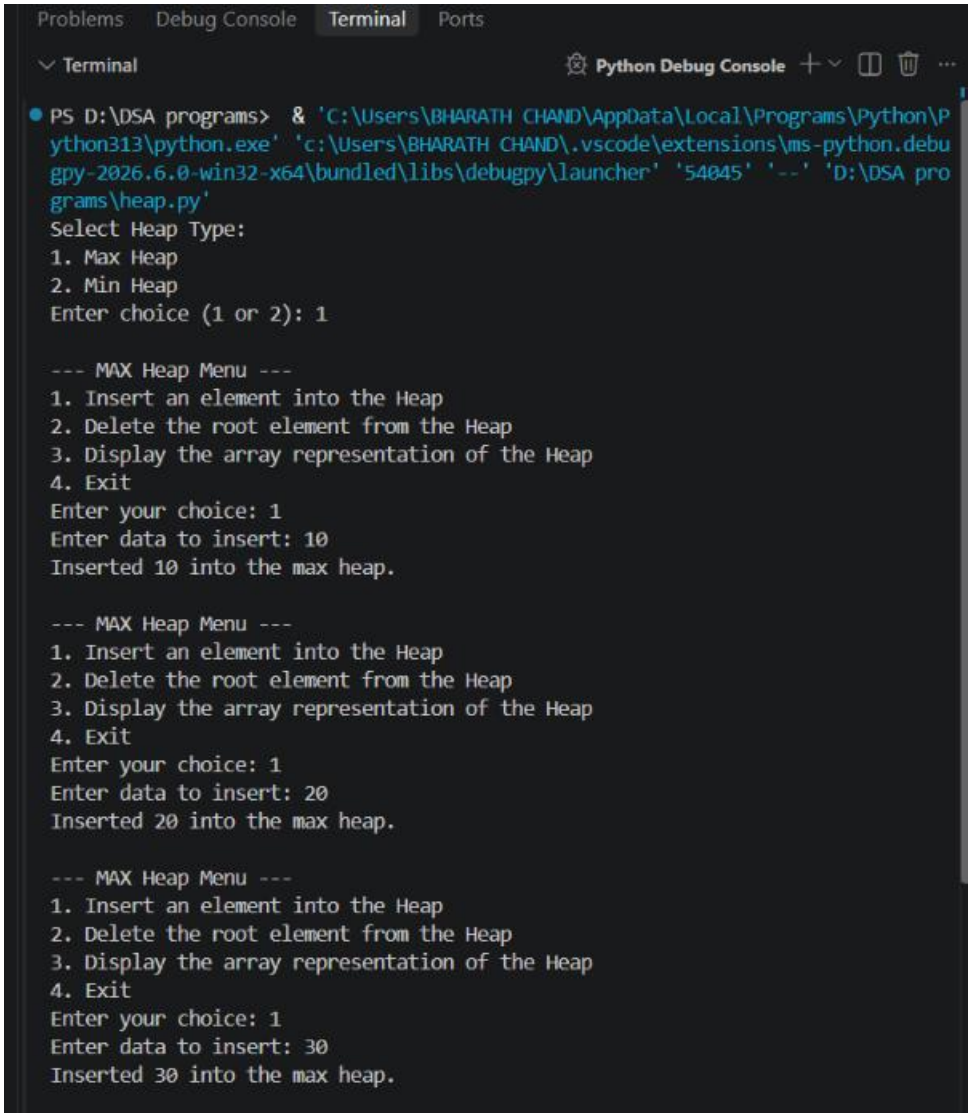
heap = BinaryHeap(h_type)
while True:
    print(f"\n--- {h_type.upper()} Heap Menu ---")
    print("1. Insert an element into the Heap")
    print("2. Delete the root element from the Heap")
    print("3. Display the array representation of the Heap")
    print("4. Exit")
```

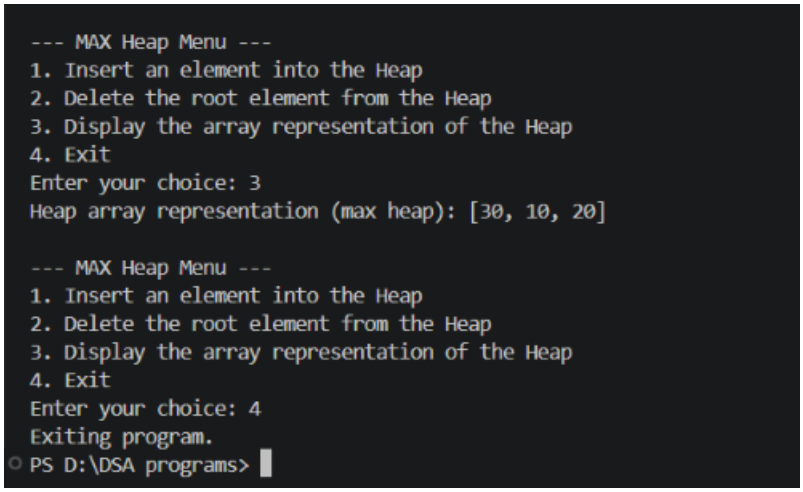
avl.py | class BinaryHeap: Untitled-1 | bst.py | countdown.py

```
    ch = int(input("Enter your choice: "))

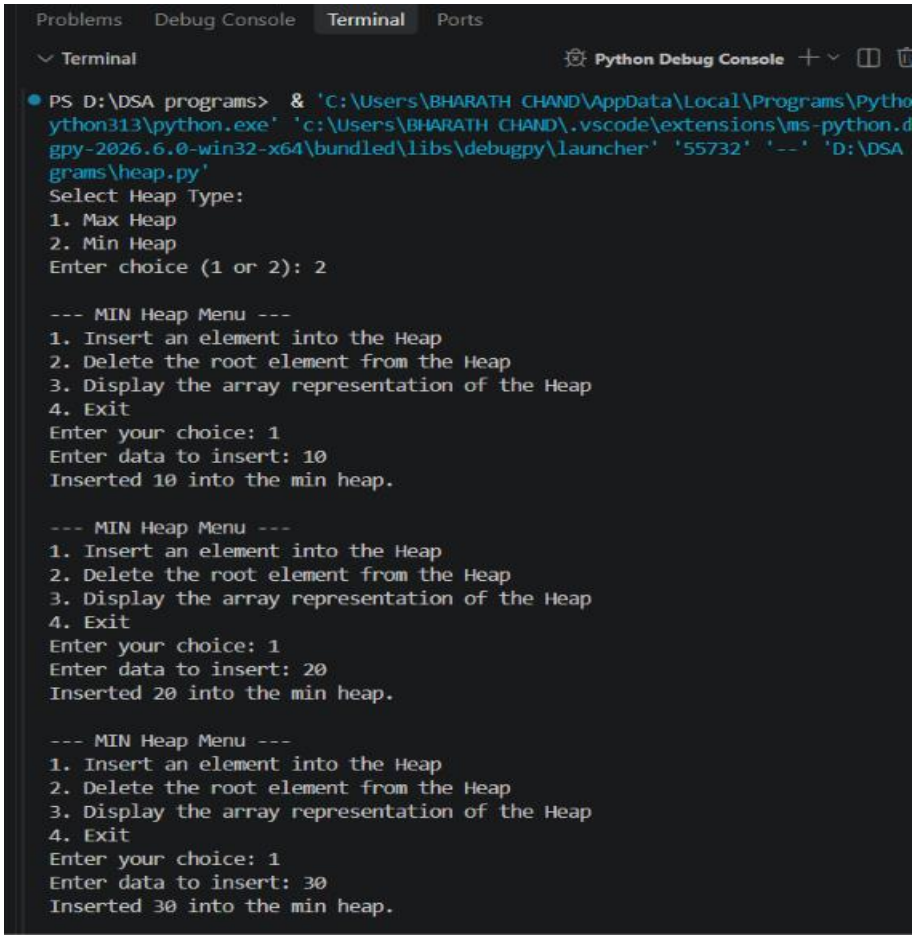
    if ch == 1:
        data = int(input("Enter data to insert: "))
        heap.insert(data)
    elif ch == 2:
        heap.delete_root()
    elif ch == 3:
        heap.display()
    elif ch == 4:
        print("Exiting program.")
        break
    else:
        print("Invalid choice! Please try again.")
```

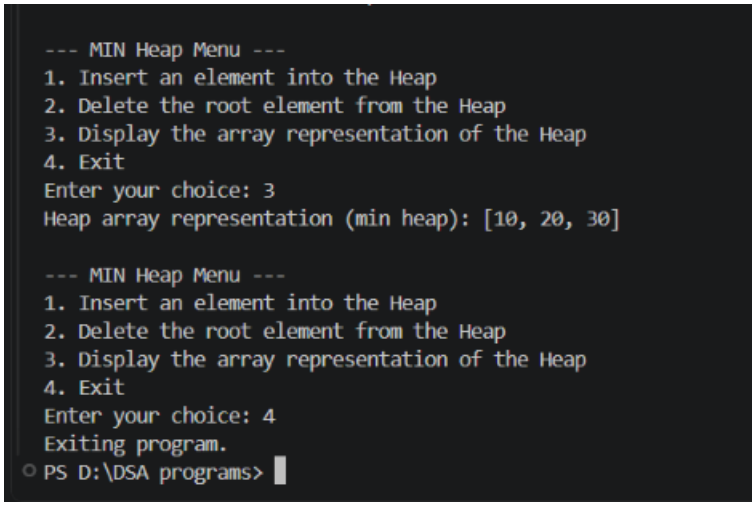
Output:


Problems
Debug Console
Terminal
Ports
Terminal
Python Debug Console
PS D:\DSA programs> & 'C:\Users\BHARATH CHAND\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\BHARATH CHAND\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '54045' '--' 'D:\DSA programs\heap.py'
Select Heap Type:
1. Max Heap
2. Min Heap
Enter choice (1 or 2): 1
--- MAX Heap Menu ---
1. Insert an element into the Heap
2. Delete the root element from the Heap
3. Display the array representation of the Heap
4. Exit
Enter your choice: 1
Enter data to insert: 10
Inserted 10 into the max heap.
--- MAX Heap Menu ---
1. Insert an element into the Heap
2. Delete the root element from the Heap
3. Display the array representation of the Heap
4. Exit
Enter your choice: 1
Enter data to insert: 20
Inserted 20 into the max heap.
--- MAX Heap Menu ---
1. Insert an element into the Heap
2. Delete the root element from the Heap
3. Display the array representation of the Heap
4. Exit
Enter your choice: 1
Enter data to insert: 30
Inserted 30 into the max heap.


--- MAX Heap Menu ---
1. Insert an element into the Heap
2. Delete the root element from the Heap
3. Display the array representation of the Heap
4. Exit
Enter your choice: 3
Heap array representation (max heap): [30, 10, 20]
--- MAX Heap Menu ---
1. Insert an element into the Heap
2. Delete the root element from the Heap
3. Display the array representation of the Heap
4. Exit
Enter your choice: 4
Exiting program.
PS D:\DSA programs>

Output:


Problems
Debug Console
Terminal
Ports
Terminal
Python Debug Console
PS D:\DSA programs> & 'C:\Users\BHARATH CHAND\AppData\Local\Programs\Pytho
ython313\python.exe' 'c:\Users\BHARATH CHAND\.vscode\extensions\ms-python.d
gpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55732' '--' 'D:\DSA
grams\heap.py'
Select Heap Type:
1. Max Heap
2. Min Heap
Enter choice (1 or 2): 2
--- MIN Heap Menu ---
1. Insert an element into the Heap
2. Delete the root element from the Heap
3. Display the array representation of the Heap
4. Exit
Enter your choice: 1
Enter data to insert: 10
Inserted 10 into the min heap.
--- MIN Heap Menu ---
1. Insert an element into the Heap
2. Delete the root element from the Heap
3. Display the array representation of the Heap
4. Exit
Enter your choice: 1
Enter data to insert: 20
Inserted 20 into the min heap.
--- MIN Heap Menu ---
1. Insert an element into the Heap
2. Delete the root element from the Heap
3. Display the array representation of the Heap
4. Exit
Enter your choice: 1
Enter data to insert: 30
Inserted 30 into the min heap.


--- MIN Heap Menu ---
1. Insert an element into the Heap
2. Delete the root element from the Heap
3. Display the array representation of the Heap
4. Exit
Enter your choice: 3
Heap array representation (min heap): [10, 20, 30]
--- MIN Heap Menu ---
1. Insert an element into the Heap
2. Delete the root element from the Heap
3. Display the array representation of the Heap
4. Exit
Enter your choice: 4
Exiting program.
PS D:\DSA programs>

WEEK-12:

AIM: Write a program to implement the following three collision resolution techniques in a Hash Table:

1. Separate Chaining

2. Linear Probing

3. Quadratic Probing

avl.py heap.py class SeparateChainingHashTable: Untitled-1 bst.py

```python
class SeparateChainingHashTable:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        return key % self.size

    def insert(self, key, value):
        index = self._hash(key)
        for kv in self.table[index]:
            if kv[0] == key:
                kv[1] = value
                print(f"Updated key {key} with value {value}.")
                return
        self.table[index].append([key, value])
        print(f"Inserted ({key}, {value}) at index {index}.")

    def search(self, key):
        index = self._hash(key)
        for kv in self.table[index]:
            if kv[0] == key:
                return kv[1]
        return None

    def display(self):
        print("\n--- Separate Chaining Hash Table ---")
        for i, bucket in enumerate(self.table):
            print(f"Index {i}: {bucket}")


class LinearProbingHashTable:
    def __init__(self, size):
        self.size = size
        self.keys = [None] * size
        self.values = [None] * size
```

avl.py | heap.py | class SeparateChainingHashTable: Untitled-1 ● | bst.py | countdown.py

```python
class LinearProbingHashTable:
    def _hash(self, key):

    def insert(self, key, value):
        initial_index = self._hash(key)
        index = initial_index
        i = 0
        while self.keys[index] is not None and self.keys[index] != key:
            i += 1
            index = (initial_index + i) % self.size
            if i == self.size:
                print("Hash Table is Full! Cannot insert.")
                return

        self.keys[index] = key
        self.values[index] = value
        print(f"Inserted ({key}, {value}) at index {index}.")

    def search(self, key):
        initial_index = self._hash(key)
        index = initial_index
        i = 0
        while self.keys[index] is not None:
            if self.keys[index] == key:
                return self.values[index]
            i += 1
            index = (initial_index + i) % self.size
            if i == self.size:
                break
        return None

    def display(self):
        print("\n--- Linear Probing Hash Table ---")
        for i in range(self.size):
            if self.keys[i] is not None:
                print(f"Index {i}: Key = {self.keys[i]}, Value = {self.values[i]}")
            else:
                print(f"Index {i}: Empty")
```

avl.py | heap.py | class SeparateChainingHashTable: Untitled-1 ● | bst.py | countdown.py

```python
class QuadraticProbingHashTable:
    def __init__(self, size):
        self.size = size
        self.keys = [None] * size
        self.values = [None] * size

    def _hash(self, key):
        return key % self.size

    def insert(self, key, value):
        initial_index = self._hash(key)
        index = initial_index
        i = 0
        while self.keys[index] is not None and self.keys[index] != key:
            i += 1
            index = (initial_index + i * i) % self.size
            if i >= self.size:
                print("Hash Table is Full or Probe Sequence Loop! Cannot insert.")
                return

        self.keys[index] = key
        self.values[index] = value
        print(f"Inserted ({key}, {value}) at index {index}.")

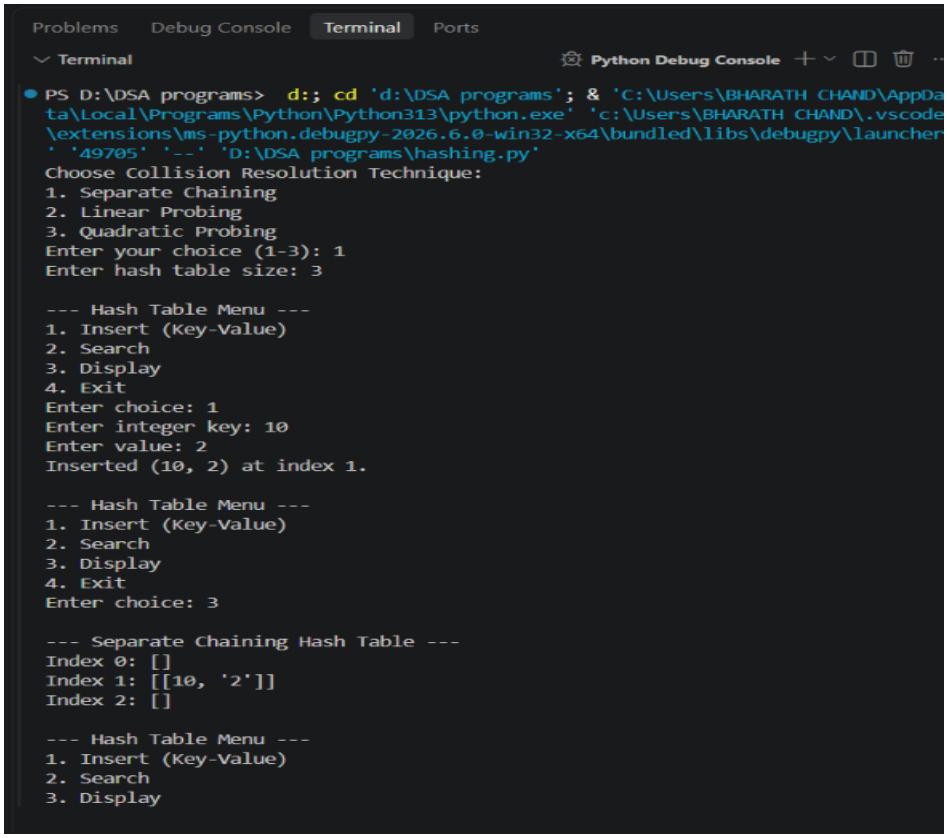
    def search(self, key):
        initial_index = self._hash(key)
        index = initial_index
        i = 0
        while self.keys[index] is not None:
            if self.keys[index] == key:
                return self.values[index]
            i += 1
            index = (initial_index + i * i) % self.size
            if i >= self.size:
                break
        return None
```

avl.py | heap.py | class SeparateChainingHashTable: Untitled-1 ● | bst.py | countdown.py

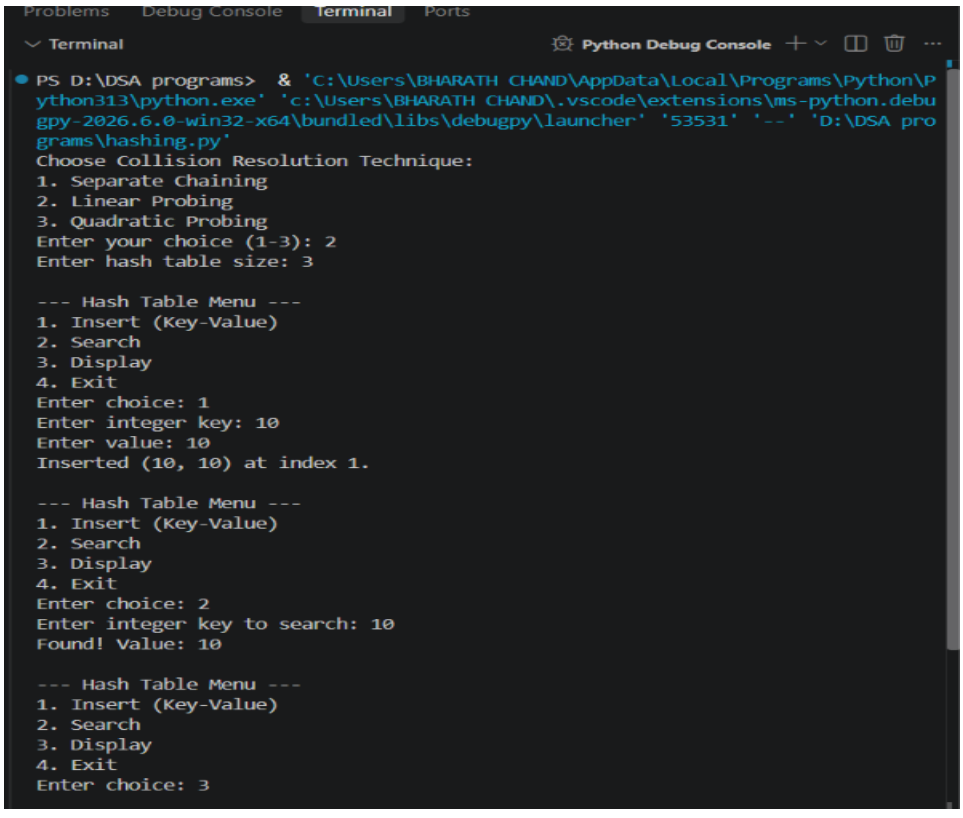
```
78  class QuadraticProbingHashTable:
115     def display(self):
116         print("\n--- Quadratic Probing Hash Table ---")
117         for i in range(self.size):
118             if self.keys[i] is not None:
119                 print(f"Index {i}: Key = {self.keys[i]}, Value = {self.values[i]}")
120             else:
121                 print(f"Index {i}: Empty")
122
123
124 # Main Menu-Driven Program
125 print("Choose Collision Resolution Technique:")
126 print("1. Separate Chaining")
127 print("2. Linear Probing")
128 print("3. Quadratic Probing")
129
130 choice = int(input("Enter your choice (1-3): "))
131 size = int(input("Enter hash table size: "))
132
133 if choice == 1:
134     ht = SeparateChainingHashTable(size)
135 elif choice == 2:
136     ht = LinearProbingHashTable(size)
137 elif choice == 3:
138     ht = QuadraticProbingHashTable(size)
139 else:
140     print("Invalid choice!")
141     exit()
142
143 while True:
144     print("\n--- Hash Table Menu ---")
145     print("1. Insert (Key-Value)")
146     print("2. Search")
147     print("3. Display")
148     print("4. Exit")
149
```

```
149
150     ch = int(input("Enter choice: "))
151     if ch == 1:
152         key = int(input("Enter integer key: "))
153         value = input("Enter value: ")
154         ht.insert(key, value)
155     elif ch == 2:
156         key = int(input("Enter integer key to search: "))
157         result = ht.search(key)
158         if result is not None:
159             print(f"Found! Value: {result}")
160         else:
161             print("Key not found.")
162     elif ch == 3:
163         ht.display()
164     elif ch == 4:
165         print("Exiting program.")
166         break
167     else:
168         print("Invalid choice! Try again.")
```

Output:



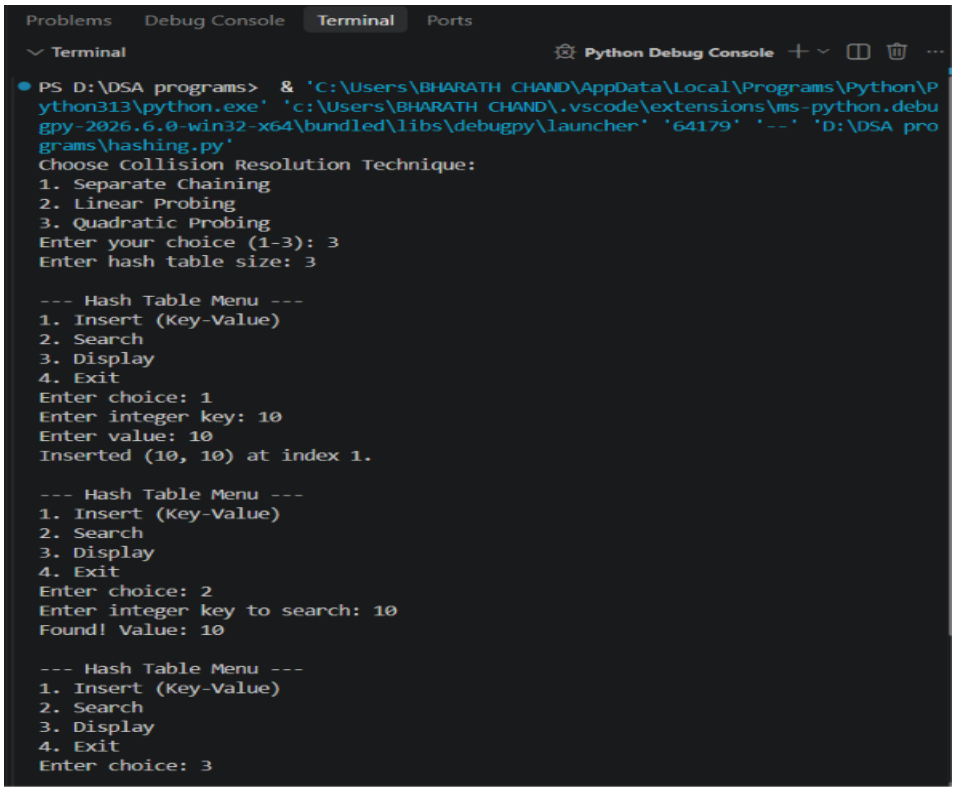
Output:



```
--- Linear Probing Hash Table ---
Index 0: Empty
Index 1: Key = 10, Value = 10
Index 2: Empty

--- Hash Table Menu ---
1. Insert (Key-Value)
2. Search
3. Display
4. Exit
Enter choice: 4
Exiting program.
PS D:\DSA programs>
```

Output:




```
--- Quadratic Probing Hash Table ---
Index 0: Empty
Index 1: Key = 10, Value = 10
Index 2: Empty

--- Hash Table Menu ---
1. Insert (Key-Value)
2. Search
3. Display
4. Exit
Enter choice: 4
Exiting program.
PS D:\DSA programs>
```